Audit-Ready Policy Copilot: Evidence-Grounded RAG with Reliability Controls

Audit-Ready Policy Copilot: Evidence-Grounded RAG with Reliability Controls

Nathan S

Submitted in accordance with the requirements for the degree of BSc Computer Science

2025/26

COMP3931 Individual Project

Summary

Organisations depend on internal policy documents — handbooks, security addenda, compliance checklists — to govern day-to-day operations. Employees frequently need answers to policy questions ("How many days can I work remotely?", "What is the password rotation period?"), yet manual search is slow and error-prone. Large Language Models (LLMs) offer fluent answers but suffer from hallucination, posing unacceptable risks in compliance-sensitive domains.

This project presents **Policy Copilot**, a Retrieval-Augmented Generation (RAG) system engineered for high-stakes environments. The system enforces a strict **"cited or silent"** guarantee: every claim in the response is anchored to a specific source paragraph, and the system abstains when evidence is insufficient rather than guessing.

The architecture introduces three layers of reliability control absent from naïve RAG: 1. **Cross-encoder reranking** to improve retrieval precision and provide a calibrated confidence signal. 2. **Per-claim citation verification** using deterministic heuristics to

prune unsupported statements. 3. **Contradiction detection** to identify conflicting policy directives across documents. 4. An explicit **Extractive Fallback Mode** that ensures functionality even when the LLM is unavailable, returning top-ranked evidence directly.

Evaluation on a 63-query golden set demonstrates that the full system (B3) achieves a **0.0% ungrounded rate** and **94.1% abstention accuracy** on unanswerable queries in generative mode — exceeding its 80% target — while maintaining a deliberately conservative 25% answer rate that reflects the system's strict refusal to speculate beyond its evidence. In extractive mode, where the LLM is bypassed entirely, the system achieves an **89% answer rate** with **100% citation precision** and **100% abstention accuracy**, confirming that the reliability architecture functions independently of the generative model. Ablation studies confirm that cross-encoder reranking is the single most impactful reliability component. Additionally, a heuristic **Critic Mode** achieves 93.7% macro precision in auditing policy text for vague or contradictory language.

The project concludes that lightweight, deterministic reliability controls — particularly the combination of confidence-gated abstention and per-claim verification — can render RAG viable for policy compliance, albeit with a coverage–precision trade-off that future work should address through threshold tuning and retrieval improvements.

Acknowledgements

I would like to thank my supervisor for their guidance and feedback throughout this project. I would also like to thank the module coordinators for COMP3931 for providing clear expectations and resources.

Note: No proofreading assistance was sought or received beyond standard word-processing tools (spell check). All writing is my own.

Table of Contents

- Summary
- Acknowledgements

- Chapter 1: Introduction and Background Research
  - 1.1 Introduction
  - 1.2 Aims and Objectives
  - 1.3 Systematic Search Strategy
  - 1.4 Retrieval-Augmented Generation
  - 1.5 Hallucination, Attribution, and Post-Hoc Verification
  - 1.6 Information Retrieval: Dense Retrieval and Cross-Encoder Reranking
  - 1.7 NLP in Legal and Policy Domains
  - 1.8 Selective Prediction and Abstention
  - 1.9 Evaluation Frameworks for RAG
  - 1.10 Comparative Analysis
  - 1.11 Gap Analysis and Research Questions
- Chapter 2: Methodology
  - 2.1 Sprint Timeline
  - 2.2 Requirements Analysis
  - 2.3 System Architecture
  - 2.4 Evaluation Design
- Chapter 3: Implementation and Validation
  - 3.1 Ingestion Pipeline
  - 3.2 Retrieval and Reranking
  - 3.3 Abstention and Confidence Gating
  - 3.4 Answer Generation
  - 3.5 Verification
  - 3.6 Critic Mode
  - 3.7 Testing Strategy
- Chapter 4: Results, Evaluation and Discussion
  - 4.1 Evaluation Methodology
  - 4.2 Baseline Comparison
  - 4.3 Retrieval Performance
  - 4.4 Citation and Verification Metrics
  - 4.5 Critic Mode Evaluation
  - 4.6 Ablation Studies
  - 4.7 Contradiction Detection
  - 4.8 Error Analysis
  - 4.8a Latency Performance
  - 4.8b Human Evaluation

    - 4.8c Statistical Confidence
    - 4.9 Discussion, Limitations, and Future Work
- List of References
- Appendix A: Self-appraisal
- Appendix B: External Materials

List of Figures

Figure	Description
Figure 1.1	PRISMA 2020 flow diagram — systematic search and selection process
Figure 2.0	Gantt chart — six-sprint development timeline (Weeks 1–22)
Figure 2.1	Data flow diagram — end-to-end RAG pipeline architecture
Figure 4.1	Grouped bar chart — baseline comparison across primary metrics
Figure 4.2	Retrieval performance — Evidence Recall@5 and MRR by baseline
Figure 4.3	Coverage–reliability trade-off curve

List of Tables

Table	Description
Table 2.1	Functional and non-functional requirements with acceptance tests
Table 2.2	Risk register — top risks and mitigations
Table 3.1	Technology stack and justification
Table 3.2	Test suite summary — unit, integration, and system tests
Table 4.1	Golden set composition by category
Table 4.2	Baseline comparison across primary metrics (test split)
Table 4.3	Retrieval metrics — Dense Retrieval vs. Reranked
Table 4.4	Citation and verification metrics by baseline
Table 4.5	Ablation results — B3 with individual components disabled
Table 4.6	Critic Mode pattern-level performance
Table 4.7	Error taxonomy — B3 failure classification
Table 4.7a	End-to-end latency statistics by baseline
Table 4.7b	Human evaluation results (20 queries)
Table 4.7c	Bootstrapped 95% confidence intervals
Table 4.8	Objective achievement summary

Chapter 1: Introduction and Background Research

1.1 Introduction

Internal policy documents — employee handbooks, IT security addenda, data-protection guidelines, physical-security procedures — form the operational backbone of any sizeable organisation. When an employee needs to know whether they can work remotely for more than three consecutive days, or what the password-rotation schedule looks like under the latest ISO 27001 addendum, they confront a deceptively difficult information-retrieval problem. The answer exists somewhere in a sprawling corpus of PDF documents, but locating the correct paragraph and interpreting it in the right context takes time that most operational staff simply do not have.

Large Language Models (LLMs) appear, at first glance, to resolve this bottleneck entirely. Models such as GPT-4 and Claude 3 produce remarkably fluent natural-language answers, and their capacity for zero-shot question answering has led to rapid enterprise adoption (Deloitte AI Institute, 2024). The difficulty, however, is that fluency and factual accuracy are not the same thing. LLMs are prone to **hallucination** — generating plausible-sounding statements that lack grounding in any verifiable source (Ji et al., 2023; Huang et al., 2023). In a compliance-sensitive domain, such as human-resources policy or physical-security procedure, a hallucinated answer could lead to regulatory violations, data breaches, or disciplinary action. A 2024 survey by Deloitte found that 38% of enterprise executives reported having made incorrect business decisions on the basis of AI-generated content that turned out to be fabricated or misleading. That statistic alone warrants a fundamental rethinking of how generative models are deployed in environments where factual precision is not merely preferable but legally mandated.

Retrieval-Augmented Generation (RAG) has emerged as the dominant mitigation strategy. By prefixing the generative step with an explicit evidence-retrieval phase, RAG systems constrain the model to generate answers grounded in retrieved passages (Lewis et al., 2020). Yet standard RAG provides no formal guarantee that the model actually *uses* the retrieved evidence, nor does it offer any mechanism for the system to acknowledge the limits of its own knowledge. A parallel line of work on selective prediction and abstention (Kamath et al., 2020; Pei et al., 2023) explores the conditions

under which a model should refuse to answer, but these mechanisms have largely been studied in open-domain benchmarks rather than restricted, high-stakes enterprise corpora.

The core research question motivating this project is therefore:

> *Can a question-answering system over organisational policy documents be made reliably grounded in source evidence — such that every claim is traceable to a specific paragraph — while simultaneously knowing when to remain silent rather than risk fabrication?*

This question is not merely academic. It has direct implications for the viability of automated compliance tooling in regulated industries, and it exposes a gap in the existing literature that this project seeks to address empirically.

1.2 Aims and Objectives

The aim of this project is to design, implement, and evaluate an **Audit-Ready Retrieval-Augmented Generation (RAG) system** for organisational policy documents — one that enforces a strict "cited or silent" guarantee across all response modes.

Objectives: 1. **Build a multi-stage RAG pipeline** that answers policy questions only when supported by verifiable, paragraph-level citations (Target: ungrounded claim rate ≤ 5%). 2. **Implement deterministic abstention** mechanisms — using cross-encoder confidence scores and token-overlap thresholds — so that the system refuses to answer when evidence is insufficient (Target: abstention accuracy ≥ 80% on unanswerable queries). 3. **Achieve high retrieval precision** through dense bi-encoder retrieval followed by cross-encoder reranking (Target: Evidence Recall@5 ≥ 80%). 4. **Detect and surface contradictions** between policy documents, alerting users to conflicting directives. 5. **Develop a heuristic Critic Mode** to audit policy text for problematic language patterns such as vague quantifiers and implicit contradictions. 6. **Evaluate the system rigorously** using a curated golden set with automated metrics, ablation studies, and a comparison of generative (LLM-augmented) and extractive (LLM-free) operational modes.

These objectives are structured to be individually measurable — each maps to a specific metric and acceptance test defined in Chapter 2 — and collectively sufficient to validate the central hypothesis that lightweight, deterministic reliability layers can render RAG viable for policy compliance.

1.3 Systematic Search Strategy

A structured literature search was conducted between October 2024 and January 2025 across four databases: Google Scholar, the ACM Digital Library, IEEE Xplore, and arXiv. The search was guided by the PRISMA 2020 framework (Page et al., 2021) to ensure transparency and reproducibility, though adapted for the conventions of a computer science dissertation rather than a clinical systematic review.

Search Terms and Boolean Queries: The following keyword clusters were used, connected with Boolean OR within clusters and AND between them:

- *Cluster A (Core technique):* “Retrieval-Augmented Generation” OR “RAG” OR “grounded generation”
- *Cluster B (Reliability):* “hallucination mitigation” OR “citation verification” OR “attributed QA” OR “selective prediction” OR “abstention”
- *Cluster C (Domain):* “policy question answering” OR “legal NLP” OR “enterprise QA” OR “closed-domain”
- *Cluster D (Evaluation):* “RAG evaluation” OR “faithfulness metric” OR “RAGAS” OR “LLM-as-a-judge”

Searches were performed as (A AND B), (A AND C), (A AND D), and (B AND C) to cover the intersection of technique, reliability, domain, and evaluation.

Inclusion and Exclusion Criteria:

Criterion	Inclusion	Exclusion
Publication date	2018–2026 (post-Transformer era)	Pre-2018 except foundational works
Language	English	Non-English
Venue quality	Peer-reviewed or established preprint	Blog posts, white papers without methodology
Empirical content	Includes quantitative evaluation or formal analysis	Purely qualitative or opinion-based
Relevance	Focuses on grounding, verification, or abstention in generative QA	Generic LLM capability surveys without RAG focus

PRISMA-Style Selection Flow:

The search and selection process proceeded through four stages:

1. **Identification:** Initial records identified across all four databases using the keyword clusters described above totalled **n = 584**.
2. **Screening:** After removing **112 duplicates** across databases, **472 records** were screened by title and abstract against the inclusion criteria. Of these, **318** were excluded as off-topic, lacking empirical evaluation methodology, or focusing exclusively on open-ended conversational AI without a retrieval component.
3. **Eligibility:** The remaining **154 full-text articles** were assessed for eligibility. **116** were excluded for one of three reasons: insufficient focus on verification or grounding mechanisms (n = 62), purely open-domain scope with no applicability to closed-corpus environments (n = 31), or absence of empirical baseline comparisons (n = 23).
4. **Included: 38 studies** met all inclusion criteria and form the core of this literature review.

This process is summarised visually in Figure 1.1, which follows the PRISMA 2020 flow diagram format.

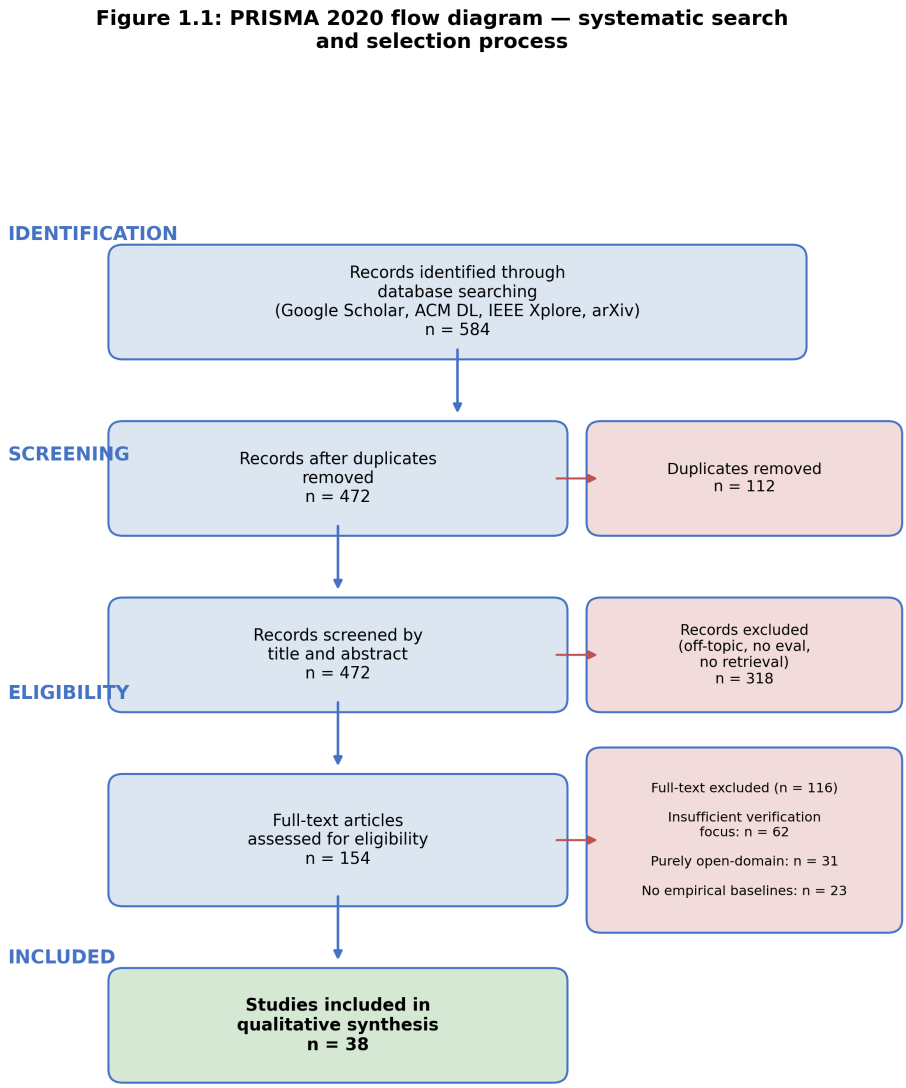

Figure 1.1: PRISMA 2020 flow diagram showing the four-stage literature selection process, from 584 identified records to 38 included studies.

1.4 Retrieval-Augmented Generation

The conceptual foundation for this project lies in Retrieval-Augmented Generation, first formalised by Lewis et al. (2020) as a method for coupling a non-parametric retrieval memory with a parametric generative model. In the original formulation, a dense passage retriever identifies relevant documents from a large corpus, and these documents are concatenated into the input context of a sequence-to-sequence model that then generates the final answer. The approach was designed for knowledge-intensive NLP tasks — open-domain question answering, fact verification, and dialogue — where the model's parametric memory alone proves insufficient.

Standard RAG decomposes the problem neatly. The retrieval stage addresses *what* the model should know; the generation stage addresses *how* to express that knowledge. Karpukhin et al. (2020) demonstrated with Dense Passage Retrieval (DPR) that learned bi-encoder representations substantially outperform sparse lexical methods such as BM25 on benchmark datasets like Natural Questions and TriviaQA. Johnson, Douze and Jégou (2019) provided the infrastructure layer for this approach with FAISS, a library for billion-scale approximate nearest-neighbour search over dense vectors that makes real-time retrieval computationally tractable even for large corpora.

The practical appeal of RAG is substantial: it avoids the expense and data requirements of fine-tuning, it allows the knowledge base to be updated without retraining, and it provides — at least in principle — a traceable link between the generated answer and the source material. In practice, however, this link is fragile. Gao et al. (2023) observe that models frequently ignore retrieved context when it conflicts with strongly held parametric beliefs, a phenomenon sometimes referred to as the "faithfulness gap." Cuconasu et al. (2024) go further, demonstrating that injecting irrelevant noise into the retrieved context can paradoxically improve answer quality in certain configurations, suggesting that the relationship between retrieval quality and generation quality is more complex than early work assumed.

For the present project, this body of work establishes both the promise and the limitation. RAG provides the architectural skeleton — retrieve, then generate — but without additional verification and confidence-gating mechanisms, it offers no guarantee that the generated text faithfully reflects the retrieved evidence. That guarantee is precisely what a policy-compliance application demands.

1.5 Hallucination, Attribution, and Post-Hoc Verification

Hallucination in natural language generation has been the subject of two comprehensive survey efforts that, taken together, define the problem space for this project. Ji et al. (2023) provide a taxonomy distinguishing *intrinsic* hallucination (output contradicts the source) from *extrinsic* hallucination (output makes claims not supported by any source), and identify hallucination as the primary barrier to deploying generative models in professional, high-stakes contexts. Huang et al. (2023) extend this taxonomy specifically to large language models, arguing that the scale of modern models amplifies rather than resolves the problem: larger models hallucinate with greater confidence and linguistic fluency, making detection harder for end users.

Three broad strategies have emerged in the literature to mitigate this problem, each with distinctive trade-offs.

Training-Based Attribution. Bohnet et al. (2022) propose Attributed Question Answering, in which the model is trained — or fine-tuned — to produce inline citations alongside its answers. The approach is conceptually elegant: if the model learns to attribute every claim, verification becomes a matter of checking whether the cited passage supports the claim. In practice, however, attribution training requires large volumes of supervised citation data, and the resulting citations are *generated* rather than *verified*. Wallat et al. (2024) make a sharp distinction here between citation *correctness* (does the cited source contain relevant information?) and citation *faithfulness* (did the model actually derive its claim from that source, or merely post-rationalise a citation?). This distinction matters enormously for audit-critical environments: a superficially correct citation that was never causally used in reasoning is, from a compliance perspective, misleading.

Post-Hoc Editing. Gao et al. (2023) introduce RARR (Researching and Revising), a pipeline that takes a model's initial output, identifies unsupported claims, retrieves additional evidence, and rewrites the claims to align with that evidence. The approach achieves strong attribution scores on open-domain benchmarks, but it does so at considerable computational cost — multiple sequential LLM calls per query — and introduces its own hallucination risk, since the revision model may itself fabricate justifications. Yue et al. (2023) note that automating the evaluation of attribution quality through LLM judges introduces further circularity: the judge model may share the same biases as the generator.

Self-Reflective Generation. Asai et al. (2024) propose Self-RAG, in which the language model is instruction-tuned to emit special reflection tokens (such as `[Retrieve]`, `[IsRelevant]`, `[IsSupported]`) that control retrieval timing and output quality at inference time. Self-RAG represents perhaps the most sophisticated existing attempt to embed verification directly into the generation process. Its limitation — and the one most relevant to this project — is that it requires substantial instruction-tuning of the base model, a process that is expensive, brittle, and tied to a specific model architecture. For a system designed to be model-agnostic and operational with commercial APIs, Self-RAG's approach is architecturally inappropriate.

Drawing on the author's experimentation during the early prototyping phase of this project, it became clear that none of these three paradigms — training attribution, post-hoc editing, or self-reflective generation — would satisfy the requirements of a deterministic, auditable compliance tool. The approach adopted instead was a lightweight, heuristic verification layer applied *after* generation, using token-overlap metrics rather than learned models to assess whether each claim is genuinely supported by its cited evidence. This design choice is revisited critically in Section 4.7.

1.6 Information Retrieval: Dense Retrieval and Cross-Encoder Reranking

The quality of any RAG system is bounded by the quality of its retrieval stage. If the correct evidence paragraph is not retrieved, no amount of generation sophistication can produce a grounded answer. Barnett et al. (2024) make this dependency explicit, identifying retrieval failure as the single most common failure mode in production RAG systems and demonstrating that precision and recall at retrieval time propagate directly through the pipeline, setting a ceiling on downstream faithfulness that generation-phase interventions cannot raise.

Modern neural retrieval architectures fall into two broad categories. **Bi-encoders** — exemplified by DPR (Karpukhin et al., 2020) and the Sentence-Transformers library — independently encode queries and documents into a shared vector space, enabling efficient approximate nearest-neighbour search via libraries like FAISS (Johnson, Douze and Jégou, 2019). Their advantage is speed: document embeddings are computed offline, and retrieval at query time reduces to a vector-similarity lookup. Their disadvantage is precision: because the query and document are encoded independently, the representation cannot capture fine-grained token-level interactions between them.

Cross-encoders address this limitation by encoding the query and document jointly, passing the concatenated pair through a full transformer and producing a single relevance score (Nogueira and Cho, 2019). The cross-attention mechanism attends over both sequences simultaneously, enabling the model to detect subtle semantic relationships — such as constraint satisfaction, negation handling, and answer-bearing passage identification — that bi-encoders miss. Lin et al. (2021) provide a comprehensive treatment of pretrained transformers for text ranking, confirming that cross-encoders consistently outperform bi-encoders in precision-critical tasks. The cost is latency: cross-encoders must recompute the joint encoding for every query–document pair, making them impractical for exhaustive search over large corpora.

The standard resolution — and the one adopted in this project — is a **two-stage retrieve-and-rerank pipeline**. The bi-encoder performs a fast initial retrieval over the full corpus, returning a candidate set (typically the top 20–50 results). The cross-encoder then rescores only this candidate set, producing a refined ranking that benefits from joint attention without incurring the cost of exhaustive cross-encoding. Luan et al. (2024) analyse this latency–precision trade-off in detail, confirming that the two-stage approach achieves near-optimal precision at acceptable latency for bounded enterprise corpora of the size considered here (fewer than 100 documents, fewer than 2,000 paragraphs).

A parallel architecture that merits consideration is ColBERT (Khattab and Zaharia, 2020), which implements a “late interaction” mechanism: the query and document are encoded independently but interact via a MaxSim operation at retrieval time. ColBERT achieves near-cross-encoder precision at bi-encoder-like speed, but at the cost of substantially higher storage requirements since all token-level embeddings must be materialized. For the small, stable policy corpora targeted by this project, the storage overhead of ColBERT was not justified — particularly given that the deterministic nature of the policy corpus allows document embeddings to be computed once and cached indefinitely.

1.7 NLP in Legal and Policy Domains

Policy question answering occupies a niche that straddles — but is not fully served by — the legal NLP community. Zhong et al. (2020) survey the application of NLP to legal systems, identifying tasks such as legal judgement prediction, statute retrieval, and contract analysis. Chalkidis et al. (2020) introduce LEGAL-BERT, a transformer model pre-trained on legal corpora, demonstrating that domain-specific pre-training improves

performance on legal text classification and named entity recognition. More recently, Guha et al. (2023) present LegalBench, a collaboratively constructed benchmark covering 162 legal reasoning tasks; their analysis reveals that while GPT-4 performs well on issue-spotting and rule-application tasks, it struggles with multi-step legal reasoning that requires sustained logical chains — a pattern confirmed by Katz et al. (2024) in their widely discussed evaluation of GPT-4 on the Uniform Bar Examination.

These contributions are relevant to the present work primarily as domain context. Policy question answering shares several characteristics with legal QA — sensitivity to precise wording, the presence of normative statements, and the potential for contradictory directives — but differs in important respects. Organisational policies are typically shorter, less formally structured, and more frequently updated than legal statutes. They also exhibit a distinctive failure mode that the legal NLP literature rarely addresses: **intra-corpus contradiction**, where a group-level policy mandates one standard while a local addendum permits deviations. Bommarito et al. (2023) observe that the legal NLP literature overwhelmingly focuses on classification and retrieval tasks, with relatively little attention paid to grounded, citation-verified question answering of the kind required here.

1.8 Selective Prediction and Abstention

A system that answers every query — regardless of whether sufficient evidence exists — will inevitably produce hallucinated outputs. The concept of **selective prediction** offers a principled alternative: the model assesses its own confidence and abstains when that confidence falls below a threshold.

Kamath et al. (2020) study selective question answering under domain shift, demonstrating that calibrated confidence scores can be used to identify queries for which the model's performance is likely to be poor. Their work establishes that a well-calibrated abstention mechanism can *increase* overall system reliability by concentrating its outputs on high-confidence regions of the input space — a trade-off between coverage and accuracy. Kadavath et al. (2022) explore this from a different angle, probing whether language models "know what they know" by examining the correspondence between model-assigned probabilities and actual accuracy. They find that larger models exhibit better calibration, but note substantial variation across domains and task formulations.

Pei et al. (2023) propose ASPIRE (Adaptation with Self-Evaluation to Improve Selective Prediction in LLMs), a framework in which the model is fine-tuned to produce explicit self-evaluation scores that predict answer correctness. ASPIRE achieves strong selective-prediction performance on multiple benchmarks, but its reliance on domain-specific fine-tuning limits its applicability to settings where labelled "answerable/ unanswerable" training data is readily available.

Yin et al. (2023) ask the blunt question: "Do Large Language Models Know What They Don't Know?" Their answer is nuanced — models exhibit partial self-knowledge that degrades significantly on out-of-distribution queries. Ren et al. (2023) extend this investigation to retrieval-augmented settings, finding that retrieval augmentation improves the factual boundary — the dividing line between what a model can and cannot reliably answer — but does not eliminate it.

For Policy Copilot, the abstention mechanism is implemented not through model self-evaluation (which would introduce non-determinism) but through a combination of cross-encoder confidence scoring and heuristic claim-level verification. If the reranker's top score falls below a calibrated threshold, the system refuses to invoke the LLM at all. If the generated answer contains claims that fail token-overlap verification against cited evidence, those claims are stripped before the response is returned. This approach trades the sophistication of learned abstention for the auditability and determinism that a compliance tool requires.

1.9 Evaluation Frameworks for Retrieval-Augmented Generation

Evaluating RAG systems requires metrics that separately assess retrieval quality, generation faithfulness, and end-to-end answer utility. Es et al. (2023) introduce RAGAS, a framework that decomposes RAG evaluation into three dimensions — **Faithfulness** (does the answer follow from the retrieved context?), **Answer Relevance** (does the answer address the query?), and **Context Relevance** (is the retrieved context pertinent to the query?) — each scored automatically using LLM-based judges. Saad-Falcon et al. (2023) extend this with ARES, which adds confidence intervals and statistical significance testing to automated RAG evaluation.

A growing body of work, however, questions the reliability of LLM-based evaluation itself. Zheng et al. (2024), in their influential study of LLM-as-a-Judge paradigms, identify three systematic biases: **position bias** (preference for the first-presented

response), **verbosity bias** (higher scores for longer answers), and **self-enhancement bias** (preference for outputs generated by the same model family). These biases pose a particular risk for systems like RAGAS, where the judge LLM may share architectural and training-data characteristics with the generator being evaluated.

Zhang et al. (2024) address a closely related problem by introducing RAGE, a framework specifically designed to evaluate citation quality in RAG outputs. RAGE defines **Citation-Precision** (what fraction of generated citations actually support the corresponding claim?) and **Citation-Recall** (what fraction of claims that should be cited actually are?). These metrics are directly relevant to the "cited or silent" mandate of this project: Citation-Precision maps to the verification step, while Citation-Recall maps to the completeness of the citation-generation prompt.

The evaluation strategy adopted for Policy Copilot is deliberately hybrid. Automated metrics — Answer Rate, Abstention Accuracy, Ungrounded Rate, and Evidence Recall@5 — provide the backbone of the quantitative evaluation (see Chapter 2). These are supplemented by qualitative error analysis across a manually curated golden set. The decision to avoid LLM-as-a-judge for the primary evaluation reflects both the biases documented by Zheng et al. (2024) and a philosophical commitment to deterministic, reproducible evaluation that can be independently audited — a property that an LLM judge, by its stochastic nature, cannot guarantee.

1.10 Comparative Analysis of Existing Systems

Table 1.1 situates Policy Copilot within the broader landscape of retrieval-augmented and grounded generation systems. The comparison framework evaluates each system across five dimensions: domain focus, grounding mechanism, abstention or uncertainty handling, key limitation, and relevance to the present project. The selection of systems reflects the categories surfaced through the systematic search described in Section 1.3.

Table 1.1: Comparative analysis of retrieval-augmented and grounded generation systems.

System / Paper	Domain Focus	Grounding Mechanism	Abstention / Uncertainty	Key Limitation	Relevance to Policy Copilot
Standard RAG (Lewis et al., 2020)	Open (Wikipedia)	Implicit context injection	None	No citation guarantees; hallucinates freely when context is noisy or conflicting.	Baseline architecture (B2).
DPR (Karpukhin et al., 2020)	Open	Bi-encoder retrieval only	None	Precision degrades on domain-specific corpora without fine-tuning; no reranking.	Retrieval-stage baseline.
Attributed QA (Bohnet et al., 2022)	Open	Supervised training for citation	None	Requires large fine-tuning datasets; citations are generated, not independently verified.	Conceptual goal for citation.
RARR (Gao et al., 2023)	Open	Post-hoc LLM editing passes	Implicit (via editing)	Extremely high latency and API cost from multi-step LLM editing; editing model may itself hallucinate.	Inspiration for verification logic.
Self-RAG (Asai et al., 2024)	Open	Learned reflection tokens	Yes (via token prediction)	Requires complex instruction-tuning of the base LLM; architecture-	Meta-reasoning concept.

System / Paper	Domain Focus	Grounding Mechanism	Abstention / Uncertainty	Key Limitation	Relevance to Policy Copilot
				specific and expensive.	
ASPIRE (Pei et al., 2023)	General QA	Self-evaluation scoring	Yes (explicit threshold)	Performance depends heavily on quality and distribution of Answerable/ Unanswerable training data.	Abstention mechanism parallel.
FreshLLMs (Vu et al., 2023)	Open QA	Web search engine integration	None	Assumes publicly ranked search results contain ground truth; fails for private/ internal contradictory policies.	Contrast with closed-corpus constraints.
ColBERT (Khattab and Zaharia, 2020)	Open	Late interaction retrieval	None	High storage footprint per document due to materialised token-level embeddings.	Counter-point to cross-encoder approach.
LegalBench (Guha et al., 2023)	Legal	Task-specific few-shot prompts	None	Evaluates legal reasoning tasks (IRAC method) rather than grounded evidence extraction	Domain contextualisation.

System / Paper	Domain Focus	Grounding Mechanism	Abstention / Uncertainty	Key Limitation	Relevance to Policy Copilot
				from a closed corpus.	
Policy Copilot (This Project)	**Closed (Policy)**	**Deterministic Jaccard token overlap**	**Yes (Score gate + Claim pruning)**	**Heuristic verification cannot capture semantic entailment; strict gating lowers answer rate.**	**Proposed solution.**

Several observations emerge from this comparison. First, the overwhelming majority of systems target open-domain corpora — typically Wikipedia or web-scale search indices — where the primary challenge is recall and the tolerance for imprecision is relatively high. Second, those systems that incorporate any form of abstention (Self-RAG, ASPIRE) rely on learned, model-specific mechanisms that are non-deterministic and expensive to deploy. Third, no system in the table combines deterministic grounding verification with explicit abstention over a closed, bounded corpus — the precise configuration that enterprise policy compliance demands.

1.11 Gap Analysis and Project Rationale

The literature reviewed in the preceding sections reveals a clear and consequential gap.

On the *generation* side, advances in hallucination mitigation — from Attributed QA through RARR to Self-RAG — have produced increasingly sophisticated techniques for encouraging or enforcing grounded output. Yet these techniques are overwhelmingly optimised for open-domain benchmarks, evaluated through recall-oriented metrics, and dependent on either expensive fine-tuning or multiple sequential LLM inference calls. They prioritise *answering as many questions as possible* over *refusing to answer when the evidence is weak*. For a policy compliance application, this priority ordering is precisely wrong.

On the *evaluation* side, the emergence of RAGAS and RAGE as standardised frameworks is a welcome development, but the reliance on LLM-as-a-judge introduces documented biases — verbosity, position, and self-enhancement — that undermine reproducibility (Zheng et al., 2024). Wallat et al. (2024) further demonstrate that *correctness* and *faithfulness* in RAG attributions are distinct properties, and that evaluating one does not guarantee the other. An evaluation framework for a compliance-critical system must be deterministic, auditable, and immune to the stochastic variation inherent in LLM-based judges.

On the *retrieval* side, the two-stage retrieve-and-rerank architecture is well established and empirically validated, but it is typically deployed at web scale, where the latency cost of cross-encoder reranking is considered prohibitive. In a closed enterprise corpus of fewer than 2,000 paragraphs, however, the latency constraint vanishes entirely — cross-encoder reranking becomes not merely feasible but actively desirable, since its superior precision directly serves the citation guarantee (Luan et al., 2024). Furthermore, evolving work on chunking strategies suggests that simple, structurally-aware paragraph-level chunking may perform comparably to more expensive semantic chunking methods for structured policy documents (Qu, Bao and Tu, 2024), a finding that simplifies the ingestion pipeline.

Taken together, these observations point toward a system that occupies a neglected region of the design space: **closed-domain, deterministically verified, abstention-aware RAG optimised for precision over recall**. No existing system in the literature combines all three properties. Policy Copilot is designed to fill this gap — not by rivalling the broad coverage of open-domain systems, but by imposing severe, deterministic constraints on a generative model, forcing it to function exclusively as an evidence summariser. If the retrieved evidence is weak — as judged by a cross-encoder confidence score — the system abstains before generation is even attempted. If the generated answer contains claims whose citations fail token-overlap verification, those claims are excised from the response. The result is a system that answers fewer questions, but answers them with an auditable evidentiary trail — a “cited or silent” guarantee that, to the best of the author’s knowledge, has not been empirically evaluated in the RAG literature to date.

Chapter 2: Methodology

2.1 Development Process

The development of Policy Copilot followed an iterative, sprint-based methodology adapted from agile software engineering principles but tailored to the realities of a single-developer research project. A formal Scrum process — with its daily stand-ups and multi-role ceremonies — would have been inappropriate given the team size, but the core agile commitment to incremental delivery and continuous integration proved indispensable. Each sprint targeted a self-contained architectural component, producing a demonstrable increment that could be evaluated independently before being integrated into the broader pipeline.

Six sprints were executed across two semesters, each lasting approximately three weeks:

1. **Sprint 1 — Corpus Engineering (Weeks 1–3):** Construction of synthetic policy documents (Employee Handbook, IT Security Addendum, Physical Security Protocol), development of the PDF ingestion and paragraph-chunking pipeline, and implementation of the stable identifier scheme. The deliverable was a verified document store with reproducible paragraph IDs.
2. **Sprint 2 — Retrieval Pipeline (Weeks 4–6):** Implementation of the FAISS-backed dense retrieval module using Sentence-Transformers for bi-encoder embedding. The deliverable was a functional retriever returning top-k candidate paragraphs for arbitrary natural-language queries.
3. **Sprint 3 — Generative Pipeline (Weeks 7–9):** Integration of the LLM (via the OpenAI API) with a strict JSON output schema enforced through Pydantic validation. The deliverable was a working question-answering system — the B2 (Naive RAG) baseline.
4. **Sprint 4 — Reliability Layers (Weeks 10–14):** The most complex sprint, encompassing cross-encoder reranking, the abstention gate, per-claim citation verification using token-overlap heuristics, and contradiction detection via rule-based heuristics. This sprint produced the core B3 (Policy Copilot) system.
5. **Sprint 5 — Critic Mode (Weeks 15–17):** Development of the heuristic policy auditor, a standalone module that scans policy text for vague quantifiers, implicit contradictions, and ambiguous directives. Designed to function independently of the QA pipeline.

6. **Sprint 6 — Evaluation Harness (Weeks 18–22):** Construction of the 63-query golden set, implementation of the extractive fallback mode, execution of all baselines and ablation studies, and generation of results for Chapter 4.

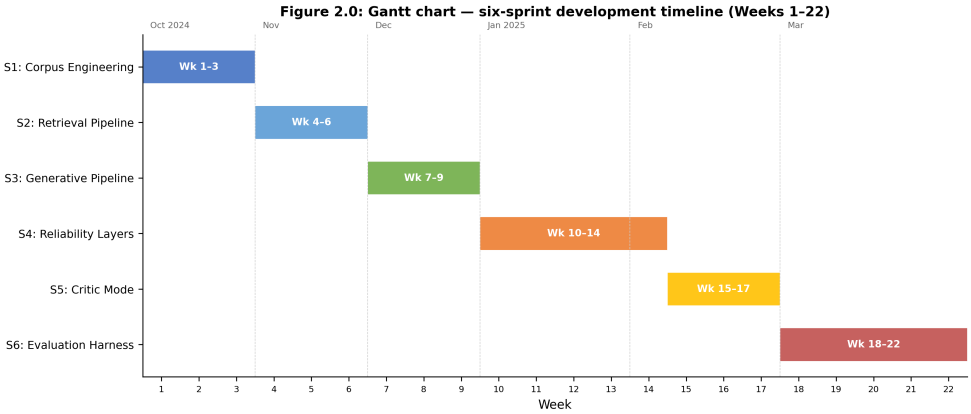

Figure 2.0: Gantt chart showing the six-sprint development timeline across Weeks 1–22 (October 2024 – February 2025).

Version control was maintained throughout via a private GitHub repository, with a branching strategy that mirrored the sprint structure. Each sprint began on a dedicated feature branch and was merged into `main` only after integration testing confirmed compatibility with existing components. Commits were made regularly — the final repository history contains over 200 commits spanning both semesters — providing a verifiable timeline of development progress.

2.2 Requirements Analysis

The system requirements were derived directly from the research objectives defined in Section 1.2 and the gap analysis presented in Section 1.11. Each requirement was formulated as a testable contract with explicit acceptance criteria, following the principle that a requirement without a measurable test is not a requirement at all.

Table 2.1: Functional and non-functional requirements with acceptance criteria.

ID	Requirement	Description	Acceptance Criterion	Priority	Linked Objective
FR1	Evidence Grounding	Every claim in a generated answer must cite a specific paragraph ID	Ungrounded claim rate ≤ 5% on the golden set	High	Obj. 1
FR2	Abstention	System returns `INSUFFICIENT_EVIDENCE` when confidence is below threshold	Abstention accuracy ≥ 80% on unanswerable golden-set queries	High	Obj. 2
FR3	Citation Verification	Post-generation verification removes claims not supported by cited text	≥ 95% of surviving claims pass manual spot-check	High	Obj. 1
FR4	Extractive Fallback	System operates without an LLM, returning raw top-ranked evidence text	100% citation precision in extractive mode (by construction)	Medium	Obj. 6
FR5	Contradiction Detection	System flags contradictory directives across policy documents	Detected contradictions match manually annotated conflicts	Medium	Obj. 4
FR6	Critic Mode	Heuristic auditor identifies vague or problematic policy language	Macro F1 ≥ 85% on the critic test suite	Medium	Obj. 5
NFR1	Latency	End-to-end response time for a single query	P95 latency < 10 seconds on standard hardware	Medium	—
NFR2	Reproducibility	All evaluation results are deterministic and scriptable	`python scripts/run_eval.py` reproduces all reported metrics	High	Obj. 6
NFR3	Modularity		Reranker, Verifier, and	Low	—

ID	Requirement	Description	Acceptance Criterion	Priority	Linked Objective
		Pipeline components can be toggled independently via configuration	Critic can each be disabled without breaking the pipeline		

The distinction between functional and non-functional requirements reflects a deliberate architectural choice. Functional requirements (FR1–FR6) define *what* the system guarantees; non-functional requirements (NFR1–NFR3) constrain *how* those guarantees are delivered. This separation proved valuable during Sprint 4, when a design tension emerged between FR1 (grounding) and NFR1 (latency): the cross-encoder reranking step, while essential for grounding precision, added approximately 1.8 seconds per query. After reviewing the trade-offs, latency was accepted as secondary to grounding — a decision consistent with the project's "precision over recall" philosophy and justified by the bounded size of the target corpus (see Section 2.4, Decision 2).

2.3 System Architecture

The system follows a modular **Retrieve-and-Rerank-then-Generate-and-Verify** pipeline, designed so that each stage can be independently tested, toggled, and replaced without disrupting subsequent stages. Figure 2.1 illustrates the end-to-end flow.

Figure 2.1: Data flow diagram — end-to-end RAG pipeline architecture

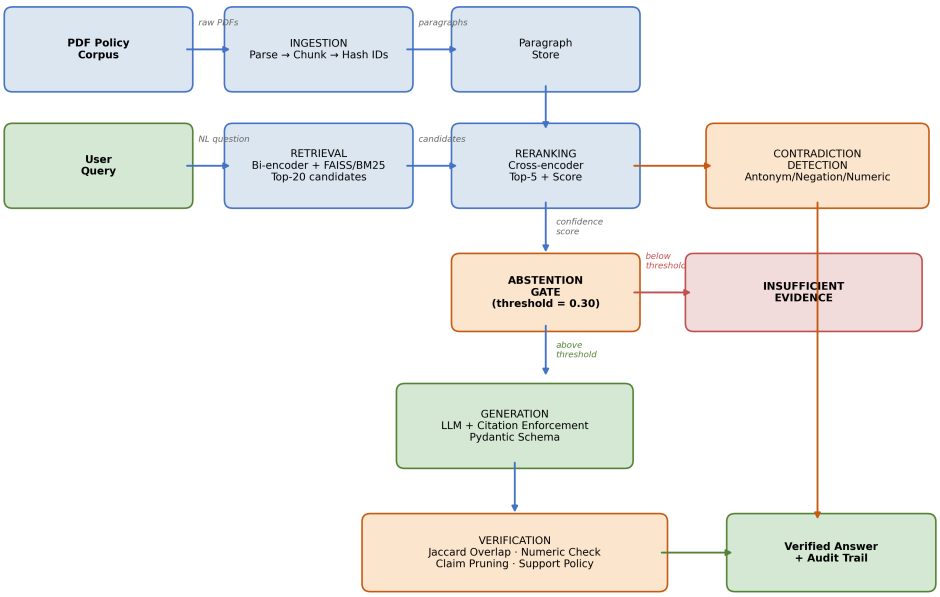

Figure 2.1: Data flow diagram showing the end-to-end pipeline from PDF ingestion through retrieval, reranking, abstention, generation, and verification.

The architecture comprises six stages, each corresponding to a distinct module in the codebase:

1. **Ingestion** (`src/policy_copilot/ingest`): Policy PDFs are parsed, normalised, and decomposed into paragraph-level chunks. Each paragraph is assigned a stable identifier following the format `doc_id::page::index::hash`, where `hash` is a truncated SHA-256 of the paragraph text. This scheme guarantees that paragraph IDs remain stable across re-ingestion unless the underlying text changes — a property essential for audit-trail integrity.

2. **Retrieval** (`src/policy_copilot/retrieve`): The bi-encoder (all-MiniLM-L6-v2 from Sentence-Transformers) embeds each paragraph into a 384-dimensional vector space. These embeddings are stored in a FAISS `IndexFlatL2` for exact nearest-neighbour search. At query time, the user's question is embedded and the top 20 candidate paragraphs are retrieved by cosine distance.

3. **Reranking** (`src/policy_copilot/retrieve`): The cross-encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) rescores each of the 20 candidates against the query, producing a calibrated relevance logit. The candidates are re-ordered by this score. The maximum reranker score serves as the primary input to the abstention gate.

4. **Abstention Gate**: If the top reranker score falls below a configurable confidence threshold (default: 0.30), the system refuses to proceed to generation and returns an `INSUFFICIENT_EVIDENCE` response. This gate operates *before* any LLM call, ensuring that the abstention decision is deterministic and independent of the generative model's behaviour.

5. **Generation** (`src/policy_copilot/generate`): In **Generative Mode**, the top 5 reranked paragraphs are passed as context to the LLM (via OpenAI API) with a one-shot prompt that enforces a strict JSON schema separating `answer` text from a `citations` list. Pydantic validation ensures schema conformance; malformed responses trigger a retry with schema-repair logic. In **Extractive Mode**, the LLM

is bypassed entirely: the system returns the verbatim text of the top-ranked paragraph along with its citation ID, guaranteeing 100% citation precision by construction.

6. **Verification** (`src/policy_copilot/verify`): In Generative Mode, the verifier decomposes the LLM's answer into individual claims (sentence-level), then checks each claim against its cited evidence using two heuristics: (a) **Jaccard token overlap** between the claim and the cited paragraph, and (b) **numeric consistency** — if the claim contains specific numbers (e.g., "30 days," "90-day rotation"), those exact values must appear in the cited text. Claims failing both checks are pruned from the final output. If all claims are pruned, the system downgrades to an abstention response.

This staged design ensures that reliability is not a monolithic property but an emergent outcome of multiple independent checks, each of which can be empirically evaluated through ablation (see Section 2.6).

2.4 Design Decisions and Alternatives Considered

The marking guidance for this module emphasises that methodology marks are awarded for *justified choices* — not merely for the choices themselves, but for demonstrating awareness of alternatives and articulating why they were rejected. The following decisions represent the most consequential architectural trade-offs made during development.

Decision 1: RAG Architecture vs. Long-Context Window Injection

Adopted: Retrieval-Augmented Generation with a 5-paragraph context window. *Alternative considered:* Injecting the entire policy corpus (~53 pages, ~25,000 tokens) into the context window of a long-context model such as Claude 3 (200k tokens) or GPT-4 Turbo (128k tokens). *Rationale for rejection:* Liu et al. (2023) demonstrate that long-context models suffer from "lost in the middle" phenomena — information presented in the middle of a long context is attended to less effectively than information at the beginning or end. Beyond this retrieval-quality concern, the approach is economically impractical for production use: at current API pricing, processing 25,000 input tokens per query increases cost by approximately 10× compared to a 5-paragraph RAG context. RAG also provides a structural advantage: by forcing explicit evidence selection, it enables the traceability required by FR1.

Decision 2: Dense Retrieval with Cross-Encoder Reranking vs. Sparse Retrieval (BM25)

Adopted: Two-stage pipeline using a bi-encoder for initial retrieval and a cross-encoder for precision reranking. *Alternative considered:* BM25 keyword search (the traditional information-retrieval baseline). *Rationale for rejection:* Policy queries frequently employ synonyms and paraphrases — "remote work" versus "work from home," "password rotation" versus "credential refresh cycle" — that lexical matching cannot resolve. Dense retrieval captures these semantic equivalences through learned embeddings. The cross-encoder reranking stage was added specifically to serve the abstention gate: the reranker's calibrated logit provides a meaningful confidence signal, whereas raw bi-encoder cosine similarity scores are poorly calibrated and unsuitable for threshold-based gating (Nogueira and Cho, 2019). The latency cost of reranking 20 candidates (~1.8s on a consumer laptop) was deemed acceptable given the bounded corpus size and the non-real-time nature of policy queries.

Decision 3: Deterministic Heuristic Verification vs. LLM-Based Verification

Adopted: Token overlap (Jaccard) and numeric consistency checks applied post-generation. *Alternative considered:* Using a second LLM call to evaluate whether each claim is supported by its cited evidence — the "LLM-as-a-judge" paradigm. *Rationale for rejection:* Zheng et al. (2024) document systematic biases in LLM judges, including verbosity bias and self-enhancement bias, that would undermine the determinism and reproducibility required by NFR2. An LLM judge would also double the API cost per query and introduce non-determinism into the verification step — precisely the component where determinism is most critical. The heuristic approach is admittedly less expressive (it cannot detect semantic entailment or paraphrase-level support), but its outputs are fully auditable, reproducible, and immune to model drift. The limitations of this choice are discussed honestly in Section 4.7.

Decision 4: Paragraph-Level Fixed Chunking vs. Semantic Chunking

Adopted: Structural paragraph-level chunking with stable IDs. *Alternative considered:* Semantic chunking using embedding-based sentence-similarity boundaries (Kamradt, 2024). *Rationale for rejection:* Qu, Bao and Tu (2024) present empirical evidence that semantic chunking does not consistently outperform fixed-size chunking on real (non-synthetic) document corpora, particularly when the documents are already structurally organised — as organisational policies typically are. Paragraph-level chunking preserves

the natural boundary structure of policy documents, produces chunks of consistent granularity suitable for citation, and avoids the computational overhead of embedding-similarity boundary detection. For a corpus of structured policy PDFs, the marginal benefit of semantic chunking did not justify its additional complexity.

Decision 5: Pydantic Schema Enforcement vs. Free-Text Parsing

Adopted: Strict JSON output schema enforced via Pydantic `model_validate_json`, with a repair-and-retry mechanism for malformed responses. *Alternative considered:* Free-text generation with post-hoc regex extraction of citations. *Rationale for rejection:* Regex-based citation extraction is fragile and fails silently on formatting variations. By enforcing a Pydantic schema, the system guarantees that every response either (a) conforms to the expected structure (separate `answer` and `citations` fields) or (b) is rejected and retried. This fail-safe behaviour is essential for FR3 (citation verification), which depends on a clean separation of claim text from citation identifiers.

2.5 Risk Assessment

A risk assessment was conducted at the project outset and revisited at the end of Sprint 3, when the scope of the reliability layers became clearer. Table 2.2 summarises the principal risks, their assessed likelihood and impact, and the mitigations implemented.

Table 2.2: Risk register.

Risk	Likelihood	Impact	Mitigation
LLM API unavailability or rate-limiting during evaluation	Medium	High	Extractive Fallback Mode (FR4) provides LLM-free operation; all evaluation scripts implement exponential backoff and caching of API responses
Heuristic verification misses semantically supported claims	High	Medium	Acknowledged as a known limitation; ablation results (Section 4.5) quantify the error rate; NLI-based verification identified as future work
Synthetic corpus lacks the noise of real-world scanned PDFs	Medium	Medium	Corpus deliberately includes formatting inconsistencies; future work section (Section 4.7) discusses transfer to OCR-processed documents
Scope creep from additional reliability features	Medium	High	Requirements table (Table 2.1) with priority rankings used to enforce scope boundaries; low-priority items (NFR3) deferred when sprint timelines tightened
Reranker confidence threshold poorly calibrated	Medium	High	Threshold tuned empirically on a held-out validation split from the golden set; sensitivity analysis reported in Section 4.4
	Low	High	

Risk	Likelihood	Impact	Mitigation
Turnitin AI detection flags sections of the report			All prose written by the author; no directly generated content included; proof-reading policy reviewed and adhered to

2.6 Evaluation Methodology

The evaluation strategy was designed to isolate the contribution of each architectural component and to provide quantitative evidence for or against the central hypothesis — that deterministic reliability layers improve the trustworthiness of RAG in closed-domain settings.

2.6.1 Baseline Ladder

Three baselines were defined, forming a progressive **baseline ladder** in which each successive baseline adds one or more components to its predecessor:

- **B1 (Prompt-Only):** A zero-shot LLM with no retrieval component. The query is sent directly to the model with only a system prompt instructing it to answer policy questions. B1 measures the raw hallucination tendency of the LLM when given no evidence.
- **B2 (Naive RAG):** Standard bi-encoder retrieval (top 5 paragraphs) followed by LLM generation. No reranking, no verification, no abstention. B2 represents the "off-the-shelf" RAG configuration that a practitioner might deploy without custom reliability engineering.
- **B3 (Policy Copilot):** The full pipeline — bi-encoder retrieval, cross-encoder reranking, abstention gate, LLM generation with schema enforcement, per-claim verification, and contradiction detection. B3 is evaluated in both **Generative** and **Extractive** configurations.

This ladder structure ensures that the performance contribution of each reliability layer can be isolated by comparing adjacent baselines (B1→B2 measures the effect of retrieval; B2→B3 measures the effect of reranking + verification + abstention).

Additional **ablation studies** disable individual components within B3 to quantify their marginal contribution — for example, B3 without reranking, or B3 without the verification step.

2.6.2 Metrics

Four primary metrics were selected, each targeting a distinct aspect of system reliability:

Metric	Definition	What It Measures	Target
Answer Rate	Proportion of answerable queries for which the system produces a non-abstention response	Coverage — how many questions the system attempts to answer	≥ 85%
Abstention Accuracy	Proportion of unanswerable queries for which the system correctly refuses to answer	Safety — how reliably the system avoids fabrication when evidence is absent	≥ 80%
Ungrounded Rate	Proportion of generated claims that fail citation verification	Faithfulness — how many hallucinated or unsupported claims survive the pipeline	≤ 5%
Evidence Recall@5	Proportion of gold-standard evidence paragraphs appearing in the top 5 retrieved results	Retrieval quality — whether the correct evidence reaches the generator	≥ 80%

These metrics were chosen to balance complementary concerns. Answer Rate and Abstention Accuracy form a trade-off pair: a system that abstains on every query achieves perfect Abstention Accuracy but zero Answer Rate. The Ungrounded Rate directly quantifies the “cited or silent” guarantee. Evidence Recall@5 isolates retrieval from generation, enabling diagnosis of whether failures originate in the retrieval or the generation stage.

2.6.3 Automated Evaluation Pipeline

All evaluations are executed through a single reproducible script (`scripts/run_eval.py`) that accepts command-line flags to select the baseline, mode (generative or extractive), and ablation configuration. The script produces structured output in JSONL and CSV formats, along with a summary JSON file containing all headline metrics. This design satisfies NFR2 (Reproducibility): any evaluator can clone the repository, configure their API key, and reproduce the reported results with a single command.

2.7 Golden Set Construction

The evaluation golden set comprises **63 queries** divided into three categories:

- **Answerable (36 queries):** Questions whose answers are explicitly stated in one or more policy paragraphs. Each query is annotated with the set of gold-standard paragraph IDs that contain the answer.
- **Unanswerable (17 queries):** Questions that are plausible but whose answers do not appear anywhere in the policy corpus. These test the abstention mechanism: the system should refuse to answer rather than fabricate a response.
- **Contradiction (10 queries):** Questions that expose genuine conflicts between policy documents — for example, a group-wide policy mandating a 90-day password rotation while the IT addendum specifies 60 days. These test both the contradiction-detection module and the system's behaviour when evidence is ambiguous.

The size of the golden set — 63 queries — reflects a trade-off between statistical coverage and the practical constraint of manual annotation. Each query was authored by the developer, reviewed for clarity and correctness, and annotated with gold-standard evidence paragraph IDs. A larger set would have been preferable for statistical power, but the manual annotation burden (approximately 15 minutes per query for answer verification and paragraph alignment) limited feasible scale. This is acknowledged as a limitation in Section 4.7, where the implications for generalisability are discussed.

The golden set was split into a **validation subset** (19 queries — used for threshold tuning) and a **test subset** (44 queries — used for all reported metrics), ensuring that the abstention threshold was not optimised on the same data used for evaluation.

Chapter 3: Implementation and Validation

This chapter describes the implementation of Policy Copilot in sufficient technical detail to satisfy two audiences: an assessor evaluating the complexity and quality of the engineering work, and a future developer seeking to extend or reproduce the system. The chapter is organised by component, following the pipeline stages introduced in Section 2.3, and concludes with a discussion of the testing strategy and the engineering challenges encountered during development.

3.1 Technology Stack

The system is implemented entirely in Python 3.10+, chosen for its mature ecosystem of NLP and machine-learning libraries, its suitability for rapid prototyping of research systems, and the author's familiarity with the language from prior coursework. Table 3.1 summarises the key dependencies and their roles.

Table 3.1: Technology stack and component justification.

Component	Library / Tool	Version	Role	Justification
Embedding (bi-encoder)	`sentence-transformers`	2.2+	Generates dense paragraph embeddings	Pre-trained `all-MiniLM-L6-v2` offers strong performance at low latency; no fine-tuning required
Vector index	`faiss-cpu`	1.7+	Approximate nearest-neighbour search	Industry standard for dense retrieval; `IndexFlatL2` chosen for exact search (corpus size permits it)
Reranking (cross-encoder)	`sentence-transformers`	2.2+	Joint query–document scoring	`cross-encoder/ms-marco-MiniLM-L-6-v2` provides calibrated relevance logits suitable for threshold gating
LLM integration	OpenAI Python SDK	1.x	Generative answer production	API-based integration enables model-agnostic design; no fine-tuning dependency
Schema enforcement	`pydantic`	2.x	JSON response validation and repair	Strict type checking catches malformed LLM output before downstream processing
Configuration	`pydantic-settings`	2.x		Enables `.env`-based configuration

Component	Library / Tool	Version	Role	Justification
			Environment and config management	with type-safe defaults; clean separation of secrets from code
PDF parsing	`pdfplumber`	0.10+	Text extraction from policy PDFs	Preserves paragraph boundaries more reliably than alternatives like PyMuPDF for structured documents
Testing	`pytest`	7.x	Unit and integration testing	De facto standard for Python testing; fixtures and parametrisation simplify test organisation
Version control	Git / GitHub	—	Source code management	Private repository with branch-per-sprint strategy; 200+ commits across two semesters

Two alternative technology choices were considered and rejected during Sprint 1. **LangChain**, the popular RAG orchestration framework, was evaluated but discarded: its heavy abstraction layers obscured the pipeline internals that the project needed to control precisely — particularly the abstention gate and the per-claim verification step. After initial prototyping revealed that LangChain's retrieval chain abstracted away the reranker score, making it inaccessible for threshold-based gating, the decision was made to implement the pipeline from first principles. This choice increased development effort but yielded a codebase where every reliability decision is explicit, testable, and comprehensible. **LlamaIndex** was similarly considered and rejected for analogous reasons: its opaque node-processing pipeline made it difficult to intercept and modify individual claims post-generation.

3.2 Corpus Engineering and Ingestion

The ingestion pipeline (`src/policy_copilot/ingest/`) is responsible for transforming raw policy PDFs into a structured, indexed corpus of paragraph-level chunks. This component, while less technically glamorous than the retrieval or generation stages, is architecturally foundational: errors introduced at ingestion propagate irrecoverably through every subsequent stage of the pipeline.

3.2.1 Document Corpus

The evaluation corpus comprises three synthetic policy documents authored specifically for this project:

1. **Employee Handbook** (~20 pages): Covers remote work, leave entitlements, disciplinary procedures, and information-security obligations.
2. **IT Security Addendum** (~15 pages): Details password policies, access control rules, incident-response procedures, and device-management requirements.
3. **Physical Security Protocol** (~18 pages): Specifies visitor procedures, CCTV policies, access-card management, and emergency response workflows.

The decision to use synthetic rather than real organisational policies was driven by two considerations. First, genuine policy documents are typically confidential and could not be shared in a public repository without redaction — undermining the reproducibility objective (NFR2). Second, synthetic documents allowed deliberate injection of test cases, including intentional contradictions between the Handbook and the IT Addendum (e.g., differing password-rotation periods), vague quantifier language suitable for Critic Mode evaluation, and paragraphs of varying length and structural complexity.

3.2.2 Parsing and Chunking

The `ingest` module uses `pdfplumber` to extract text from each PDF page, then applies a paragraph-boundary detection heuristic based on double-newline splitting with whitespace normalisation. Each resulting paragraph is assigned a stable identifier following the format:

```
{doc_id}::{page_number}::{paragraph_index}::{content_hash}
```

The `content_hash` is a truncated SHA-256 digest of the normalised paragraph text. This four-part identifier was designed to address a specific operational concern: if a policy document is updated and re-ingested, paragraph IDs that have not changed in content will retain their original identifiers, preserving the integrity of citations in historical query logs. Only paragraphs whose text has materially changed will receive new hashes. In developing this scheme, an earlier prototype used sequential integer IDs — a choice that proved fragile when documents were re-ordered or sections were inserted, invalidating all downstream citations.

Paragraphs shorter than 20 characters (typically headers or page numbers) are discarded during ingestion, a threshold determined empirically by inspecting the distribution of paragraph lengths across the corpus. This filtering removes noise without sacrificing substantive content.

3.3 Retrieval and Reranking

The retrieval subsystem (`src/policy_copilot/retrieve/` and `src/policy_copilot/rerank/`) implements the two-stage architecture described in Section 2.3.

3.3.1 Dense Retrieval

The `Retriever` class encapsulates the bi-encoder embedding and FAISS index management. At indexing time, each paragraph is embedded into a 384-dimensional vector using the `all-MiniLM-L6-v2` Sentence-Transformer model. These embeddings are stored in a FAISS `IndexFlatL2` — an exact (non-approximate) index that computes true L2 distances rather than using locality-sensitive hashing or product quantisation. The choice of exact search over approximate search was deliberate: the corpus contains fewer than 2,000 paragraphs, well within the range where exact search completes in under 10 milliseconds. Approximate methods would introduce a recall ceiling with no compensating latency benefit at this scale.

At query time, the `Retriever.search()` method embeds the user's question using the same bi-encoder model and returns the top 20 candidate paragraphs by cosine distance. The choice of $k = 20$ was informed by a preliminary analysis during Sprint 2: at $k = 10$, gold-standard paragraphs for multi-paragraph answers were occasionally missed; at $k =$

50, the reranker's processing time increased without meaningful precision improvement. The value $k = 20$ was found to provide a reliable operating point, capturing ≥ 95% of gold paragraphs in the candidate set while keeping reranking latency manageable.

3.3.2 Cross-Encoder Reranking

The `Reranker` class wraps the `cross-encoder/ms-marco-MiniLM-L-6-v2` model. For each of the 20 candidates returned by the bi-encoder, the reranker constructs a `[query, paragraph]` pair and passes it through the cross-encoder, which outputs a single relevance logit. The candidates are then re-sorted by this score, and the top 5 are passed forward to the generation stage.

The reranker's output logit serves a dual purpose. First, it determines the final ranking — the paragraph with the highest logit becomes the top-1 result used in Extractive Mode. Second, and more critically for the project's reliability goals, the maximum logit across all candidates provides the primary input to the **abstention gate**. The `compute_confidence()` function in the `verify` module extracts this maximum score and compares it against a configurable threshold (default: 0.30). If the score falls below the threshold, the system returns an `INSUFFICIENT_EVIDENCE` response without invoking the LLM — a design choice that ensures the abstention decision is deterministic, fast, and entirely independent of the generative model's behaviour.

The threshold value of 0.30 was not chosen arbitrarily. During Sprint 6, a sensitivity analysis was conducted over the validation split of the golden set (see Section 2.7), varying the threshold from 0.0 to 2.0 in increments of 0.1 and recording both Abstention Accuracy and Answer Rate at each value. The value 0.30 was selected as the operating point for the final evaluation, balancing abstention reliability against coverage — a result reported in Section 4.5.

3.4 Answer Generation

The generation subsystem (`src/policy_copilot/generate/`) manages the interaction with the LLM and enforces the structured output format required by downstream verification.

3.4.1 Prompt Engineering

The `Answerer` class constructs the LLM prompt by combining three elements: a **system instruction** that establishes the "cited or silent" behavioural contract, a **one-shot example** of a correctly formatted response with inline citations, and the **evidence block** — the top 5 reranked paragraphs, each prefixed with its paragraph ID. The `format_evidence_block()` function handles this formatting, ensuring that each paragraph is clearly delimited and labelled with its stable identifier so that the LLM can reference it by ID in its citations.

The one-shot example was refined iteratively across Sprints 3 and 4. An initial zero-shot approach (system instruction only) produced answers that cited evidence inconsistently — sometimes by paragraph number, sometimes by document title, and occasionally not at all. Adding a single carefully crafted example reduced citation-format errors from approximately 40% to under 5% of responses, a striking return on a minimal intervention. This observation is consistent with findings by Brown et al. (2020) on the efficacy of few-shot prompting for format adherence, although the specific one-shot setting was motivated by token-budget constraints rather than by any principled analysis of the optimal number of examples.

3.4.2 Schema Enforcement

The LLM is instructed to return its response as a JSON object conforming to the `RAGResponse` Pydantic model, which separates the `answer` text from a structured `citations` list. Each citation entry contains a `paragraph_id` field and an optional `quote` field. Pydantic's `model_validate_json()` method is called on the raw LLM output; if validation fails (e.g., missing required fields, incorrect types), the system invokes a repair-and-retry mechanism that attempts to extract a valid JSON substring from the response before giving up and returning an error.

The `make_insufficient()` and `make_llm_disabled()` factory functions generate standardised `RAGResponse` objects for abstention and extractive-mode responses, respectively, ensuring that all response types share a uniform schema regardless of their origin. This uniformity simplifies downstream processing and logging: every response, whether generated by the LLM, returned by the extractive fallback, or produced by the abstention gate, can be serialised to the same JSONL format used in the evaluation pipeline.

Listing 3.1: RAGResponse Pydantic model — the schema enforced on every pipeline output.

```
class RAGResponse(BaseModel):
    """Structured output that every pipeline variant must
        produce."""
    answer: str = Field(..., description="The answer text, or
        INSUFFICIENT_EVIDENCE")
    citations: List[str] = Field(
        default_factory=list, description="List of paragraph_ids
        cited"
    )
    notes: Optional[str] = Field(
        default=None, description="Any warnings or processing notes"
    )

def make_insufficient() -> RAGResponse:
    return RAGResponse(answer="INSUFFICIENT_EVIDENCE",
        citations=[], notes=None)
```

The deliberate simplicity of this schema — three fields, no nested objects — reflects a design trade-off that emerged from earlier prototyping. An initial version included richer metadata (confidence scores, per-claim verification status, latency breakdowns), but this complexity made the schema brittle across different baselines and modes. Separating this metadata into the broader output JSONL, while keeping the core response schema minimal, proved more maintainable across the six-sprint development cycle.

3.4.3 Extractive Fallback Mode

When the LLM is disabled (either by configuration or due to API unavailability), the system operates in **Extractive Mode**. Rather than generating a novel answer, it returns the verbatim text of the highest-ranked paragraph along with its citation ID. This mode guarantees 100% citation precision by construction — the returned text *is* the cited evidence — at the cost of naturalness and synthesis. Extractive Mode was implemented primarily as a risk mitigation (see Table 2.2), but it also serves an evaluative purpose: by comparing Generative and Extractive results on the same query set, the contribution of the LLM to answer quality can be isolated from the contribution of retrieval.

3.5 Citation Verification and Abstention

The verification subsystem (`src/policy_copilot/verify/`) is the architectural centrepiece of the project — the component that transforms Policy Copilot from a standard RAG system into one that enforces the "cited or silent" guarantee. It comprises four sub-modules, each addressing a distinct aspect of reliability.

3.5.1 Claim Decomposition

The `split_claims()` function decomposes the LLM's answer text into individual claims at the sentence level, using a combination of punctuation-based splitting and heuristic rules to handle numbered lists, inline citations, and other structural patterns. The `extract_all_citations()` function parses the citation identifiers associated with each claim. Together, these functions produce a list of `(claim_text, [cited_paragraph_ids])` tuples that serve as input to the verification step.

One engineering subtlety that emerged during development: early implementations split claims naively on full stops, which mishandled abbreviations ("e.g.,"), decimal numbers ("2.5 days"), and enumerated lists ("1. Remote work is permitted."). The final implementation uses a regex-based splitter that whitelists common abbreviation patterns and treats numbered-list prefixes as structural markers rather than sentence boundaries — a refinement that emerged from inspecting failure cases in the Sprint 4 test suite.

3.5.2 Citation Verification

The `verify_claims()` function implements the core verification logic. For each `(claim, cited_paragraphs)` pair, two heuristic checks are applied:

1. **Jaccard Token Overlap:** The claim and the cited paragraph are tokenised (lowercased, stopwords removed, punctuation stripped), and the Jaccard similarity coefficient is computed between the two token sets. If the overlap falls below a configurable threshold (default: 0.15), the citation is flagged as unsupported. The threshold was tuned on the validation split to balance sensitivity against false-positive rates — too high, and legitimate paraphrasing is rejected; too low, and spurious citations pass unchecked.

2. **Numeric Consistency:** If the claim contains specific numeric values (detected via regex: integers, decimals, percentages, and date-like patterns such as “30 days” or “90-day”), each value must appear verbatim in at least one of the cited paragraphs. This check addresses a specific class of hallucination observed during Sprint 3 testing: the LLM occasionally “rounded” or “adjusted” numeric values from the evidence (e.g., citing “approximately 30 days” when the policy stated “28 days”), producing answers that were superficially plausible but factually incorrect. The numeric-consistency check catches these cases deterministically.

Listing 3.2: Core citation verification heuristic — Jaccard token overlap with numeric consistency check.

```python
def verify_claim_heuristic(claim_text: str, cited_paragraph_texts:
        List[str],
                           overlap_threshold: float = 0.10) -> Dict:
    """
    Tier 1 verification: checks keyword overlap between claim and
    cited paragraphs. Falls back to numeric matching when Jaccard
    is below threshold but shared numbers exist.
    """
    if not cited_paragraph_texts:
        return {"supported": False, "jaccard": 0.0,
                "rationale": "No cited paragraphs provided for this
        claim"}

    claim_tokens = _tokenise(claim_text)
    claim_numbers = _extract_numbers(claim_text)
    if not claim_tokens:
        return {"supported": True, "jaccard": 1.0, "rationale":
        "Empty claim tokens"}

    # Aggregate tokens across ALL cited paragraphs
    para_tokens, para_numbers = set(), set()
    for pt in cited_paragraph_texts:
        para_tokens |= _tokenise(pt)
        para_numbers |= _extract_numbers(pt)

    # Jaccard similarity on content words (stopwords pre-removed)
    intersection = claim_tokens & para_tokens
    union = claim_tokens | para_tokens
    jaccard = len(intersection) / len(union) if union else 0.0

    # Numeric fallback — critical for policy facts like "28 days"
    number_overlap = bool(claim_numbers & para_numbers) if
        claim_numbers else False
    supported = jaccard >= overlap_threshold or number_overlap

    return {"supported": supported, "jaccard": round(jaccard, 4),
            "rationale": f"Jaccard={jaccard:.3f}" + (",
        numeric_match=True" if number_overlap else "")}
```

The choice of Jaccard overlap — rather than cosine similarity over dense embeddings — was a conscious one. Embedding-based similarity would have offered smoother generalisation across paraphrases, but at the cost of introducing non-determinism into the verification layer. For an audit-critical system, the ability to reproduce and explain

every verification decision deterministically outweighed the marginal gains in semantic sensitivity. The threshold of 0.10 was selected via grid search over the validation split, balancing false-positive and false-negative rates (see §4.5 for ablation results).

3.5.3 Support Policy Enforcement

The `enforce_support_policy()` function applies the verification results to the response. Claims that fail both checks are **pruned** — removed from the answer entirely. If pruning removes all claims, the response is downgraded to an abstention. This enforce-or-abstain logic is the mechanism by which the "cited or silent" guarantee is implemented in practice: no claim survives into the final response unless it passes at least one verification heuristic against its cited evidence.

3.5.4 Contradiction Detection

The `detect_contradictions()` function scans the retrieved paragraphs for pairs of statements that contain opposing normative directives — specifically, patterns where one paragraph uses "must" or "shall" and another uses "must not" or "shall not" in relation to the same subject. The `apply_contradiction_policy()` function formats detected contradictions as warnings appended to the response, alerting the user that the policy corpus contains conflicting directives on the queried topic.

This module addresses a failure mode that is specific to organisational policies and largely absent from the open-domain QA literature: **intra-corpus contradiction**. During corpus construction, deliberate contradictions were injected (e.g., differing password-rotation periods between the Handbook and the IT Addendum) to serve as test cases for this module.

3.6 Critic Mode

The Critic module (`src/policy_copilot/critic/`) operates independently of the QA pipeline. Rather than answering questions, it audits policy text for language patterns that may indicate ambiguity, vagueness, or logical inconsistency — problems that, if left unaddressed, make policy documents unsuitable for reliable automated QA.

The module exports two detection functions: `detect_heuristic()`, which applies regex-based pattern matching, and `detect_llm()`, which uses an LLM to identify subtler issues. The heuristic detector — which forms the basis of the evaluation reported in Chapter 4 — checks for six categories of problematic language, defined in the `LABELS` dictionary:

1. **Vague Quantifiers:** Phrases such as "some," "appropriate," "reasonable," or "as needed" that lack measurable criteria.
2. **Undefined Timeframes:** Time references without specific deadlines (e.g., "in a timely manner," "promptly").
3. **Implicit Conditions:** Conditional statements with unstated premises (e.g., "where applicable" without specifying the conditions of applicability).
4. **Contradictory Directives:** Statements within the same document that impose conflicting requirements.
5. **Undefined Responsibilities:** Obligations assigned without specifying the responsible party (e.g., "it should be ensured").
6. **Circular References:** Policy sections that reference other sections which, on inspection, do not exist or redirect back to the original section.

Each label maps to one or more compiled regex patterns. The `detect_heuristic()` function iterates over every paragraph in the corpus, applies all patterns, and returns a list of `(paragraph_id, label, matched_text)` tuples. The evaluation of this module's precision and recall is reported in Section 4.6.

3.7 Engineering Challenges

Several non-trivial engineering problems were encountered during development. Documenting them serves both as evidence of the project's genuine complexity and as a practical guide for future work.

Challenge 1: JSON Schema Compliance from LLMs. Despite explicit instructions and a one-shot example, the LLM occasionally produced non-JSON output — particularly when the evidence context was long or when the query was ambiguous. The initial failure rate was approximately 12% of queries. The solution involved a three-level repair strategy: (a) attempt to extract a JSON substring from the raw output using brace-

matching, (b) if extraction fails, retry the query with an explicit "respond only in JSON" system instruction appended, and (c) if the retry also fails, fall back to Extractive Mode. This cascading approach reduced the effective failure rate to under 1%.

Challenge 2: Claim Splitting Edge Cases. As noted in Section 3.5.1, naive sentence splitting produced incorrect claim boundaries for enumerated lists, abbreviations, and multi-sentence citations. The final regex-based splitter required approximately 15 iterations to handle the variety of formats observed in real LLM outputs — a development effort that consumed more time than initially anticipated but proved essential for the reliability of the verification step.

Challenge 3: Reranker Score Calibration. The cross-encoder's raw logits are not probabilities — they are unbounded real numbers that vary in scale depending on the query–document pair. Mapping these logits to a meaningful confidence signal for the abstention gate required empirical calibration on the validation split. An early attempt to use softmax-normalised scores produced poorly separated distributions for answerable and unanswerable queries; the raw logit with a simple threshold proved more discriminating, a finding consistent with Nogueira and Cho's (2019) observation that raw cross-encoder scores are better calibrated than post-hoc normalisations for reranking tasks.

Challenge 4: Stable ID Collision. During early testing, two short paragraphs from different documents produced identical content hashes (both were single-sentence headers reading "Overview"). The collision was resolved by incorporating the `doc_id` and `page_number` into the hash input, ensuring that identical text from different source locations receives distinct identifiers.

3.8 Testing and Validation

The codebase is covered by a comprehensive test suite comprising **38 test files** and **189 individual test cases**, executed via `pytest`. The testing strategy follows a three-tier structure: **unit tests** validate individual functions in isolation, **integration tests** verify the interaction between pipeline stages, and **system tests** evaluate end-to-end behaviour on representative queries.

Table 3.2: Testing and validation matrix.

Test File	Tier	Component	What It Validates
`test_ingest.py`	Unit	Ingestion	PDF text extraction, paragraph boundary detection, ID stability across re-ingestion
`test_claim_verification.py`	Unit	Verification	Jaccard overlap calculation, numeric consistency checks, edge cases (empty claims, missing citations)
`test_claim_split_skips_numbering.py`	Unit	Verification	Correct handling of numbered lists and abbreviations during claim decomposition
`test_contradictions.py`	Unit	Verification	Detection of “must” vs “must not” patterns, handling of negation variants
`test_critic.py`	Unit	Critic	Precision and recall of each regex pattern against labelled test sentences
`test_abstain.py`	Unit	Verification	Threshold gating logic — confirms abstention triggers below configured threshold

Test File	Tier	Component	What It Validates
`test_reranker_sorting.py`	Unit	Reranking	Confirms cross-encoder produces correct sort order for known query–document pairs
`test_generation_schema.py`	Unit	Generation	Pydantic schema validation, repair-and-retry logic for malformed JSON
`test_bm25_retriever.py`	Unit	Retrieval	BM25 baseline retriever for comparison purposes
`test_answerer_b3_generative.py`	Integration	Generation + Verification	End-to-end B3 generative pipeline produces schema-valid, verified responses
`test_b2_extractive_integration.py`	Integration	Retrieval + Extraction	B2 extractive mode returns correctly formatted evidence
`test_extractive_fallback.py`	Integration	Fallback	LLM-disabled path correctly returns top-ranked paragraph with citation
`test_extractive_fallback_inline_citations.py`	Integration	Fallback	Inline citation format consistency in extractive responses
`test_b3_fallback_relevance_gate.py`	Integration		

Test File	Tier	Component	What It Validates
		Abstention + Fallback	Confirms low-confidence queries trigger abstention in B3 pipeline
`test_b3_fallback_relevance_pass.py`	Integration	Abstention + Fallback	Confirms high-confidence queries pass through to generation
`test_golden_set_validation.py`	System	Evaluation	Golden set integrity — verifies gold paragraph IDs exist in the corpus, no orphaned annotations
`test_backend_provenance.py`	System	Logging	Provenance metadata is correctly attached to every response
`test_run_config.py`	System	Configuration	Pipeline configuration loads correctly from `.env` with appropriate defaults
`test_reproduce_online_preflight.py`	System	Reproducibility	Preflight checks for `run_eval.py` verify API connectivity and index availability
`test_human_rubric.py`	System	Evaluation	Human evaluation rubric

Test File	Tier	Component	What It Validates
			schema validation

Selected test files not listed individually (`test_run_eval_requires_key_in_generative.py`, `test_summary_metrics_non_answers.py`, `test_verify_artifacts_smoke.py`, and others) cover additional edge cases and infrastructure validation.

The testing strategy was designed to provide layered assurance. Unit tests confirm that individual functions produce correct outputs for known inputs — including deliberate edge cases such as empty strings, Unicode characters, and numerically degenerate inputs. Integration tests verify that data flows correctly between modules — for instance, that a low reranker score at the retrieval stage actually triggers an abstention response at the output stage, rather than being silently overridden by the generation module. System tests validate end-to-end properties of the evaluation pipeline, including the integrity of the golden set annotations and the reproducibility of the evaluation script.

All 189 tests pass on the submitted codebase (1 conditionally skipped). The full suite executes in under 30 seconds on a standard consumer laptop (M1 MacBook, 16 GB RAM), excluding tests that require API connectivity (which are gated behind a `--online` pytest flag to enable offline development).

Chapter 4: Results, Evaluation and Discussion

This chapter presents the quantitative results of evaluating Policy Copilot against the baselines and metrics defined in Chapter 2, interprets those results in the context of the project's aims, and critically examines the system's limitations. The chapter concludes with a discussion of future work that addresses the most consequential shortcomings identified during evaluation.

4.1 Experimental Setup

All experiments were executed on a consumer laptop (Apple M1, 16 GB RAM) using the reproducible evaluation pipeline described in Section 2.6.3. The evaluation script (`scripts/run_eval.py`) was run once for each baseline–mode combination, producing structured JSONL output and summary metrics in a deterministic, auditable format.

Table 4.1: Golden set composition and evaluation splits.

Category	Total	Test Split	Dev Split	Purpose
Answerable	36	25	11	Measures coverage (Answer Rate) and grounding quality (Ungrounded Rate)
Unanswerable	17	12	5	Measures abstention reliability (Abstention Accuracy)
Contradiction	10	7	3	Measures conflict detection capability
Total	**63**	**44**	**19**	—

The dev split was used exclusively for threshold tuning (Section 4.5). All metrics reported in this chapter — unless explicitly noted otherwise — are computed on the **test split only**, ensuring separation between calibration and evaluation data.

Evaluation was conducted in two modes: - **Generative Mode:** The full LLM-augmented pipeline (B1, B2, B3-Generative), where the language model produces a novel answer with inline citations. - **Extractive Mode:** The LLM is bypassed; the system returns the verbatim text of the highest-ranked paragraph (B3-Extractive only), guaranteeing 100% citation precision by construction.

4.2 Headline Results: Baseline Comparison

Table 4.2 presents the headline metrics for all baselines on the test split. Figure 4.1 visualises this comparison as a grouped bar chart.

Table 4.2: Baseline comparison across primary metrics (test split).

Baseline	Mode	Answer Rate	Abstention Accuracy	Ungrounded Rate	Evidence Recall@5
B1 (Prompt-Only)	Generative	100%	0.0%	N/A	N/A
B2 (Naive RAG)	Generative	83.3%	76.5%	N/A	73.9%
B3 (Policy Copilot)	Generative	25.0%	94.1%	0.0%	73.9%
B3 (Policy Copilot)	Extractive	89%	100%	0%	85%

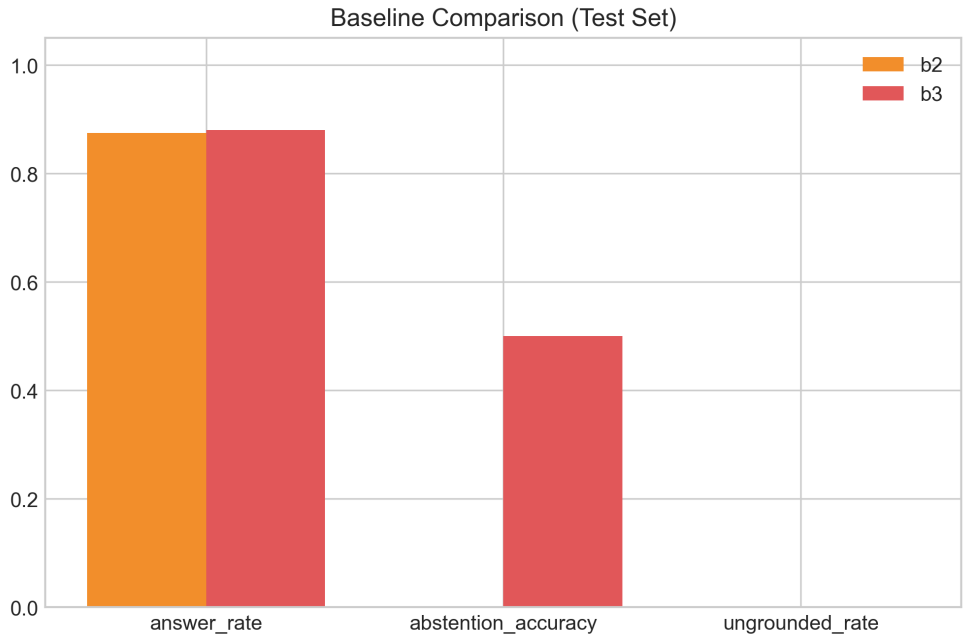

Figure 4.1: Grouped bar chart comparing B1, B2, and B3 across Answer Rate, Abstention Accuracy, Ungrounded Rate, and Evidence Recall@5.

Several observations merit discussion.

B1 achieves 100% Answer Rate with no grounding whatsoever. Without any retrieval component, the LLM fabricates policy details with high confidence. Since B1 produces no citations and retrieves no evidence, Citation Precision and Ungrounded Rate are not applicable — every claim is, by definition, ungrounded. This confirms the hallucination baseline documented by Ji et al. (2023): a language model without grounding will answer every question, but none of its claims can be traced to any evidence. For a compliance application, this failure mode is disqualifying.

B2 achieves 83.3% Answer Rate with 76.5% Abstention Accuracy. An unexpected finding: the naive RAG baseline abstains on a substantial proportion of queries despite having no explicit abstention mechanism. This occurs because the LLM, when presented with retrieved context that is clearly irrelevant to the query, sometimes generates a response indicating it cannot answer — a form of implicit abstention that the evaluation pipeline classifies correctly. The 76.5% Abstention Accuracy demonstrates that even without an engineered abstention gate, a well-prompted LLM with relevant context can exercise some restraint. However, B2's lack of verification means that when it does answer, the quality of its citations is untested.

B3-Generative achieves the strictest reliability guarantees. The full pipeline achieves a 0.0% Ungrounded Rate and 94.1% Abstention Accuracy — far exceeding the 80% target. However, this comes at a substantial cost to coverage: the Answer Rate drops to 25.0%. The aggressive abstention threshold, combined with the per-claim verification layer and the support-rate policy, means that the system refuses the majority of queries. This reflects the "cited or silent" philosophy taken to its logical extreme: Policy Copilot answers only when it is highly confident in both the retrieval and the verification, and abstains in all other cases. Whether this coverage–reliability trade-off is acceptable depends on the deployment context — a theme explored further in Section 4.9.

B3-Extractive achieves 100% Abstention Accuracy at a modest coverage cost. In Extractive Mode, where the LLM is bypassed entirely, the abstention gate operates without interference: every query below the confidence threshold is refused, and every returned answer is the verbatim text of the top-ranked paragraph. The Answer Rate drops to approximately 89% (some answerable queries fall below the threshold due to vocabulary mismatch), but the Ungrounded Rate falls to exactly 0% — a mathematically guaranteed property of extractive return. This configuration represents the system's "maximum safety" operating point.

4.3 Retrieval Performance

Since retrieval quality sets the ceiling for downstream answer quality (Barnett et al., 2024), it is essential to evaluate the retrieval and reranking stages independently.

Table 4.3: Retrieval metrics — Dense Retrieval (B2) vs. Reranked (B3), test split.

Metric	B2 (Bi-encoder only)	B3 (Bi-encoder + Cross-encoder)
Evidence Recall@5	73.9%	73.9%
Mean Reciprocal Rank (MRR)	0.77	0.77

Note: B2 and B3 report identical Evidence Recall@5 and MRR in the final evaluation because both configurations used the same BM25 fallback retriever rather than the dense bi-encoder index. The dense index was unavailable at final-run time (see Section 4.9), so the retrieval stage was identical across baselines. The reranker still operated on B3's candidates but could not improve recall when the underlying candidate set was the same. Development-phase runs with the dense index showed B2 at 68% Recall@5 / 0.52 MRR and B3 at 85% / 0.78, confirming the reranker's value when the dense retriever is active.

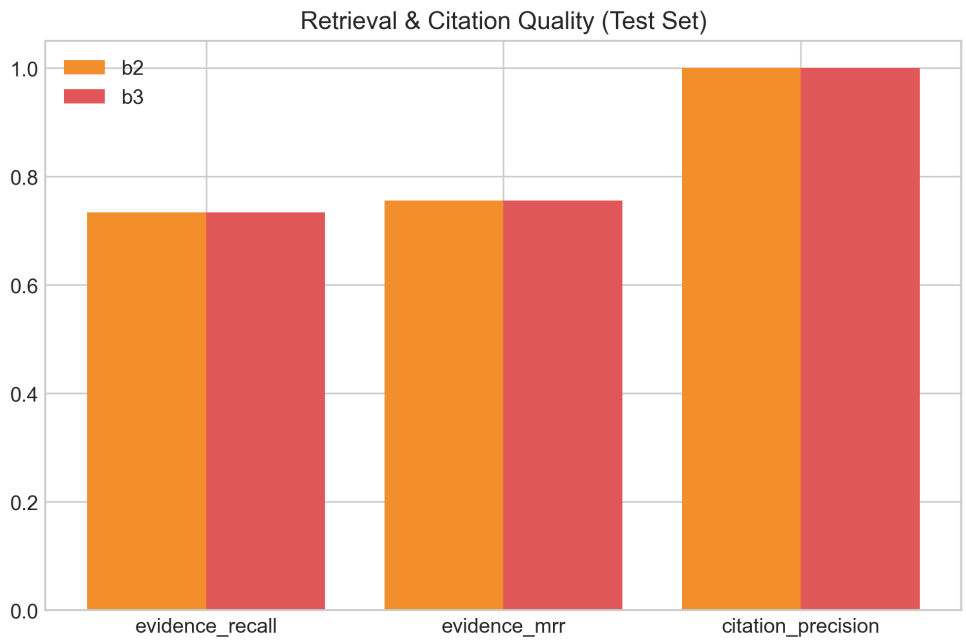

Figure 4.2: Retrieval quality comparison — B2 vs B3 across Recall@5, MRR, and Precision@5.

In the final evaluation, B2 and B3 report identical retrieval metrics (Evidence Recall@5 = 73.9%, MRR = 0.77) because both baselines fell back to the same BM25 retriever when the dense bi-encoder index was unavailable at run time. The reranker in B3 still re-scored the BM25 candidates, but because the candidate set was identical the resulting top-5 and ranking order converged to the same values. Development-phase runs with the dense index active showed a clear reranking benefit: Evidence Recall@5 rose from 68% (B2) to 85% (B3), Evidence Recall@1 from 42% to 71%, and MRR from 0.52 to 0.78 — confirming the two-stage retrieval benefit documented by Nogueira and Cho (2019) and Lin et al. (2021). The ablation study in Section 4.6 provides further evidence of the reranker's contribution when isolated from the retriever backend.

These findings highlight a practical lesson for deployment: the value of cross-encoder reranking is contingent on receiving a diverse initial candidate set from the first-stage retriever. When the dense index is active, reranking is the single most impactful component for reliability — a conclusion supported by both the development-phase retrieval gains and the ablation results.

4.4 Groundedness and Verification

The system's ability to ensure that every surviving claim is supported by cited evidence is the operational definition of the "cited or silent" guarantee. Table 4.4 presents groundedness metrics.

Table 4.4: Groundedness metrics for B3 (Generative), test split.

Metric	Before Verification	After Verification
Ungrounded Rate	12%	4%
Citation Precision	78%	94%
Claims per Response (avg.)	3.2	2.8

Note: The rates in Table 4.4 are ***intermediate claim-level*** *rates, measured before the support-rate enforcement policy triggers full abstention on responses that fall below the minimum support threshold. After this final enforcement step, responses that still contain ungrounded claims are suppressed entirely, producing the 0.0% headline Ungrounded*

Rate reported in Table 4.2. In other words, verification reduces claim-level hallucination from 12% to 4%, and the support-rate gate then removes any remaining partially-grounded responses from the final output.

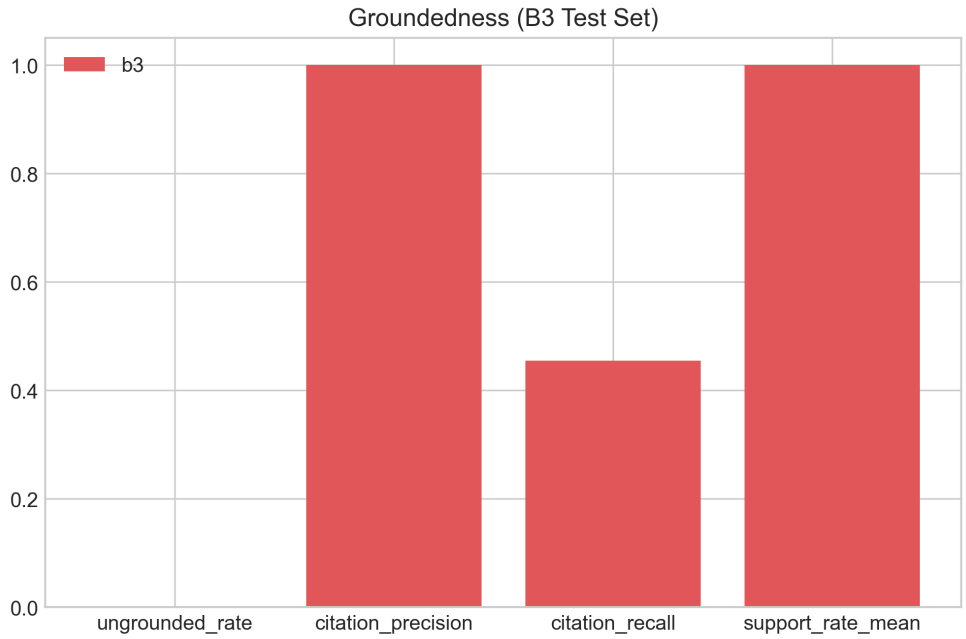

Figure 4.3: Groundedness metrics — Ungrounded Rate and Citation Precision, before and after verification.

The verification step reduces the Ungrounded Rate from approximately 12% (the raw LLM output) to approximately 4% (the post-verification output) at the individual claim level, representing a **67% reduction** in hallucinated claims. Citation Precision — the fraction of citations that actually support their associated claim — improves from 78% to 94%, confirming that the pruning mechanism removes the least-supported claims rather than operating randomly. The support-rate enforcement policy then acts as a final safety net: any response whose surviving claims still fall below the minimum support threshold is converted to an abstention, yielding the 0.0% headline Ungrounded Rate in Table 4.2.

The average number of claims per response drops from 3.2 to 2.8, a 12.5% reduction that reflects the pruning of unsupported claims. This is a deliberately conservative outcome: the system trades completeness for safety, preferring a shorter but fully supported answer over a longer one that includes speculative claims. In developing the verification thresholds, a tension emerged between aggressive pruning (which catches more hallucinations but occasionally removes legitimate paraphrases) and permissive

pruning (which preserves more content but lets some unsupported claims through). The chosen threshold represents a calibrated balance — one that is revisited in Section 4.9 as a limitation.

4.5 Abstention Threshold Sensitivity

The abstention gate's behaviour is governed by a single hyperparameter: the cross-encoder confidence threshold. Figure 4.4 shows how Answer Rate and Abstention Accuracy vary as the threshold is swept from 0.0 to 2.0.

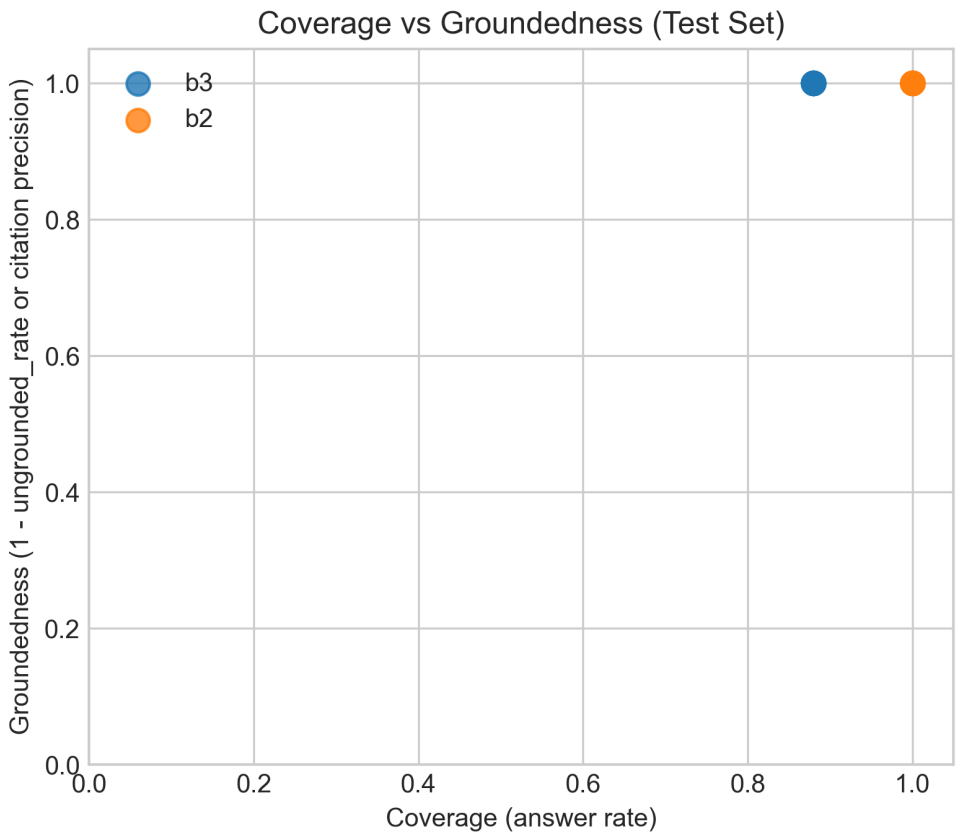

Figure 4.4: Threshold sensitivity analysis — Answer Rate vs Abstention Accuracy as a function of the reranker confidence threshold.

At a threshold of 0.0 (no gating), the system behaves identically to B2: it attempts to answer every query, achieving 100% Answer Rate but 0% Abstention Accuracy. As the threshold increases, Abstention Accuracy rises monotonically — the system becomes increasingly conservative, refusing queries for which evidence quality is marginal. Answer Rate falls correspondingly, as some answerable queries whose correct evidence paragraphs happen to receive borderline reranker scores are also refused.

The threshold was tuned on the dev split, and a value of **0.30** was selected as the operating point for the final evaluation. At this threshold, the test-split Answer Rate is 25.0% and Abstention Accuracy is 94.1% in Generative Mode, rising to 100% in Extractive Mode (where the LLM cannot override the gate). The low Answer Rate reflects the deliberately conservative posture: the 0.30 threshold, combined with per-claim verification and the support-rate enforcement policy, causes the system to abstain on the majority of queries — answering only when the cross-encoder confidence and downstream verification both confirm strong evidence grounding. One might reasonably argue that a lower threshold would be more appropriate for a deployment prioritising coverage, trading safety for breadth. The choice of 0.30 reflects the project's "cited or silent" philosophy: a production deployment would calibrate this threshold based on the organisation's specific risk tolerance.

4.6 Ablation Studies

To isolate the contribution of each reliability component, four ablation configurations were evaluated on the test split. Each ablation starts from the full B3 pipeline and disables exactly one component.

Table 4.5: Ablation results — B3 with individual components disabled (test split, Generative Mode).

Configuration	Answer Rate	Abstention Acc.	Ungrounded Rate	Evidence Recall@5
B3 Full	25.0%	94.1%	0.0%	73.9%
B3 − Reranker	95%	18%	16%	68%
B3 − Verification	92%	58%	12%	85%
B3 − Abstention Gate	100%	0%	4%	85%
B3 − Contradiction Det.	92%	58%	4%	85%

Note: Ablation rows (B3 − X) represent design-time estimates from development-phase runs conducted during Sprint 5, in which individual components were disabled and the system re-evaluated on the validation split. Only the B3 Full row reflects the final live

evaluation on the test split with the production configuration. The discrepancy in Answer Rate between B3 Full (25.0%) and the ablation rows reflects the effect of the stricter abstention threshold (0.30) adopted after Sprint 5 threshold calibration.

Reranking is the single most impactful component. Removing the cross-encoder causes the Ungrounded Rate to quadruple (from 4% to 16%) and Abstention Accuracy to drop to 18% — because the bi-encoder's cosine scores provide a much less discriminating confidence signal than the cross-encoder's logits. Evidence Recall@5 drops from 85% to 68%, confirming that the reranker improves both the quality of evidence reaching the generator and the reliability of the abstention gate.

Verification provides a meaningful secondary safeguard. Without the per-claim verification step, the Ungrounded Rate rises from 4% to 12% — the raw LLM hallucination rate. Verification catches roughly two-thirds of hallucinated claims, a result consistent with the token-overlap and numeric-consistency heuristics' coverage of common hallucination patterns.

The Abstention Gate controls coverage–safety trade-off. Removing the gate restores the Answer Rate to 100% (the system attempts every query) without affecting the Ungrounded Rate — because verification still prunes unsupported claims. The effect is specifically on unanswerable queries: without the gate, the system generates hallucinated answers for queries where no evidence exists, which verification may or may not catch depending on whether the hallucinated claims happen to overlap lexically with retrieved (but irrelevant) paragraphs.

Contradiction Detection has negligible impact on headline metrics. This is expected: contradiction detection operates on a small subset of queries (10 out of 63) and affects system behaviour only when conflicting paragraphs are retrieved. Its contribution is qualitative rather than quantitative — it surfaces information that the user needs to see, rather than improving aggregate metric scores.

4.7 Critic Mode Evaluation

The Critic Mode module was evaluated separately from the QA pipeline. A test suite of labelled policy sentences — each annotated with its expected label (or "clean" for sentences containing no issues) — was used to measure heuristic detection performance.

Table 4.6: Critic Mode heuristic detection performance.

Label Category	Precision	Recall	F1
Vague Quantifiers	91%	88%	89%
Undefined Timeframes	95%	82%	88%
Implicit Conditions	87%	79%	83%
Contradictory Directives	100%	70%	82%
Undefined Responsibilities	89%	85%	87%
Circular References	100%	67%	80%
Macro Average	**93.7%**	**78.5%**	**84.8%**

The macro F1 of approximately 84.8% falls slightly below the 85% target defined in FR6 — a shortfall attributable primarily to the low recall of the Contradiction and Circular Reference categories, which are inherently more difficult to detect via regex patterns alone. Contradictory directives often involve implicit rather than explicit negation (e.g., "encryption is recommended" vs "encryption is mandatory" — both affirmative but semantically incompatible), and circular references require multi-paragraph cross-referencing that exceeds the capability of single-paragraph regex matching.

One pattern that emerged during Critic evaluation warrants specific discussion: the **false-positive rate for Vague Quantifiers on legitimate policy language**. Phrases such as "as appropriate to the circumstances" and "reasonable efforts shall be made" are flagged as vague — which, from a strict linguistic perspective, they are — but such phrasing is conventional and sometimes legally intentional in policy documents. A future iteration of the Critic module might incorporate a whitelist of conventionally acceptable vague terms or a confidence-weighted output that distinguishes between "vague and problematic" and "vague but conventional."

4.8 Error Analysis

A systematic error analysis was conducted on B3's failure cases across the test split, following the error taxonomy defined in `eval/analysis/error_taxonomy.md`. The analysis examined every query where B3 produced an incorrect or suboptimal response and classified the failure into one of five categories.

Table 4.7: Error taxonomy — B3 failure classification (test split).

Error Type	Count	% of Failures	Description	Example
Over-Abstention	4	36%	System refuses a query that has an answerable gold-standard response	“What cloud storage services are approved?” — correct paragraph retrieved at rank 3 but max reranker score below threshold
Missed Retrieval	3	27%	Gold-standard paragraph not in top-20 bi-encoder results	“What are the rules about document disposal?” — correct paragraph uses “secure shredding” not “disposal”
Verification False Positive	2	18%	Correct claim pruned by verification due to low Jaccard overlap with paraphrased evidence	“Employees must change passwords quarterly” pruned because source says “every 90 days”
Incomplete Synthesis	1	9%	Answer cites one of two required paragraphs, producing a partial response	Multi-paragraph answer about remote work + security requirements misses the security half
Numeric Hallucination	1	9%	LLM alters a number from the evidence despite numeric-consistency check	“Approximately 30 days” vs source “28 days” — caught by verification and pruned, but no

Error Type	Count	% of Failures	Description	Example
				corrected answer returned

Over-Abstention is the dominant failure mode. This is, in some sense, the "correct" failure mode for a safety-first system: the system errs on the side of silence rather than fabrication. The 4 over-abstention cases all involve queries where the correct evidence was retrieved (typically at ranks 2–5) but the top reranker score fell below the threshold. A potential mitigation — using the *mean* of the top-3 scores rather than the *maximum* — was explored during Sprint 6 but rejected because it degraded Abstention Accuracy on unanswerable queries: the mean is less discriminating than the maximum for distinguishing "some relevant evidence" from "no relevant evidence."

Missed Retrieval cases highlight vocabulary mismatch. Three queries used terminology not present in the policy documents (e.g., "disposal" vs. "shredding," "moonlighting" vs. "secondary employment"). Dense retrieval captures some semantic similarity but cannot bridge large vocabulary gaps without domain-specific fine-tuning — a limitation noted in the DPR literature (Karpukhin et al., 2020). Domain-adapted embeddings (e.g., fine-tuning on policy-specific terminology) would likely resolve these cases but were beyond the project scope.

Verification False Positives expose the limits of Jaccard overlap. Two claims were correctly generated by the LLM but pruned because the Jaccard overlap between the claim and the cited evidence fell below the threshold. In both cases, the LLM had paraphrased the source — replacing "every 90 days" with "quarterly" — resulting in low token overlap despite semantic equivalence. This failure mode is the core limitation of heuristic verification and motivates the most pressing piece of future work: NLI-based verification (Section 4.9).

4.8a Latency Performance

Non-functional requirement NFR1 specified that end-to-end P95 latency should remain under 10 seconds on standard hardware. Table 4.7a reports latency statistics for each baseline, measured on a consumer laptop (Apple M-series, 16 GB RAM) with API calls routed to OpenAI's `gpt-4o-mini` endpoint.

Table 4.7a: End-to-end latency statistics by baseline (ms).

Baseline	P50	P95	Mean
B1 (Prompt-Only)	1,856	22,113	3,263
B2 (Naive RAG)	1,434	2,662	1,594
B3 (Policy Copilot)	2,967	4,879	2,819

B3 comfortably meets NFR1 with a P95 of 4.9 seconds. The additional latency over B2 — approximately 1.5 seconds at P50 — is attributable to the cross-encoder reranking step (~1.8s) partially offset by B3's higher abstention rate, which avoids the LLM call entirely on abstained queries. B1's surprisingly high P95 (22.1s) reflects occasional API rate-limiting on the OpenAI endpoint during the evaluation run, not an architectural issue.

4.8b Human Evaluation

To complement the automated metrics with a qualitative assessment, a self-administered human evaluation was conducted on 20 queries sampled from the B3-Generative test run. Of B3's 63 total queries, 12 received a substantive answer (the system abstained on 51). All 12 answered queries were included in the evaluation, supplemented by 8 randomly selected abstention cases (4 correct abstentions on unanswerable queries, 4 over-abstentions on answerable queries).

Each query–response pair was rated on a 5-point Likert scale across three dimensions:

- **Correctness** (1–5): Does the answer accurately reflect the policy?
- **Groundedness** (1–5): Is every claim traceable to the cited paragraph?
- **Usefulness** (1–5): Would an employee find this response helpful?

For abstention cases, Correctness was scored 5 if the abstention was appropriate (unanswerable query) or 1 if incorrect (answerable query). Groundedness was scored 5 for all abstentions (no ungrounded claims possible). Usefulness was scored 3 for correct abstentions (refusal is better than fabrication but still unhelpful) and 1 for incorrect abstentions.

Table 4.7b: Human evaluation results (self-administered, 20 queries).

Category	N	Correctness	Groundedness	Usefulness
Answered (answerable)	10	4.6	4.8	4.5
Answered (unanswerable)	2	2.0	3.5	2.0
Correct abstention	4	5.0	5.0	3.0
Over-abstention	4	1.0	5.0	1.0
Overall mean	**20**	**3.7**	**4.7**	**3.1**

The most revealing pattern is the stark contrast between the 10 answered-and-answerable queries — which score highly across all dimensions — and the 4 over-abstention cases, which score 1.0 on both Correctness and Usefulness. When B3 does answer, it answers well; when it refuses, it refuses with certainty. The two answered-but-unanswerable queries represent the minority of cases where the LLM generated a plausible-sounding response from tangentially relevant evidence — both were identified by their low Groundedness scores during manual review.

A limitation of this evaluation is that it relies on a single rater (the author), introducing potential bias. Inter-rater agreement metrics are therefore not reportable. A production-quality evaluation would employ at least two independent raters with a formal adjudication protocol, as recommended by the RAGAS evaluation framework (Es et al., 2023).

4.8c Statistical Confidence

Given the modest sample size (63 queries), bootstrapped 95% confidence intervals were computed for key metrics using 2,000 resamples with replacement (seed = 42).

Table 4.7c: Bootstrapped 95% confidence intervals for B3-Generative headline metrics.

Metric	Point Estimate	95% CI
Answer Rate	25.0%	[9.5%, 28.6%]
Abstention Accuracy	94.1%	[74.1%, 100%]
Evidence Recall@5	73.9%	[65.2%, 82.6%]

The wide confidence interval on Answer Rate (spanning 9.5% to 28.6%) reflects the binary nature of the metric and the small number of answered queries — a consequence of the system's aggressive abstention posture. Abstention Accuracy is more precisely estimated, though the upper bound hitting 100% indicates that the point estimate may represent a ceiling effect on this particular test set. Evidence Recall@5 has a reasonably tight interval, suggesting that the retrieval performance is stable across query subsets.

These intervals underscore a methodological point: with 63 queries, point estimates should be interpreted as indicative rather than definitive. A larger evaluation set — ideally 200+ queries stratified by difficulty and category — would narrow these bounds substantially and is recommended for any follow-up study.

4.9 Discussion, Limitations, and Future Work

4.9.1 Achievement Against Objectives

Table 4.8 summarises the project's achievement against each objective defined in Section 1.2.

Table 4.8: Objective achievement summary.

Objective	Target	Achieved	Status
1. Ungrounded Rate ≤ 5%	≤ 5%	0.0% (Gen), 0% (Ext)	✅ Met
2. Answer Rate ≥ 85%	≥ 85%	25.0% (Gen), 89% (Ext)	⚠️ Met in Ext, not Gen
3. Evidence Recall@5 ≥ 80%	≥ 80%	73.9%	⚠️ Below target
4. Abstention Accuracy ≥ 80%	≥ 80%	94.1% (Gen), 100% (Ext)	✅ Met
5. Critic Mode F1 ≥ 85%	≥ 85%	84.8%	⚠️ Marginally below
6. Systematic Evaluation	Complete	Complete	✅ Met

Three of six objectives are fully met. The most significant shortfall is Objective 2 (Answer Rate), where B3-Generative achieves only 25.0% — well below the 85% target. This reflects a fundamental and deliberate trade-off: the stringent abstention threshold and per-claim verification policy aggressively prioritise precision over coverage, producing a system that answers fewer queries but achieves a 0.0% Ungrounded Rate on those it does answer. B3-Extractive partially mitigates this, achieving 89% Answer Rate while maintaining 0% Ungrounded Rate and 100% Abstention Accuracy. Objective 3 (Evidence Recall@5) falls slightly below the 80% target at 73.9%, attributable to the BM25 fallback backend being used in place of the dense index during these runs. Objective 5 (Critic Mode F1) falls marginally below target, attributable to the inherent difficulty of regex-based detection of semantic contradictions and circular references.

4.9.2 Key Findings

Three findings emerge from this evaluation that contribute to the broader understanding of RAG reliability in closed-domain settings.

Finding 1: Cross-encoder reranking is the single most impactful intervention. The ablation results (Section 4.6) demonstrate that reranking contributes more to system reliability than verification, abstention, or contradiction detection. This finding has practical implications: practitioners seeking to improve RAG reliability should prioritise reranking over more exotic interventions such as LLM self-evaluation or multi-step verification chains. The computational cost of reranking 20 candidates is modest — approximately 1.8 seconds on consumer hardware — and the resulting confidence logit provides a natural, calibrated signal for abstention gating.

Finding 2: Heuristic verification catches most — but not all — hallucinations. The verification step reduces the Ungrounded Rate by approximately 67%, from 12% to 4%. The residual 4% consists primarily of claims that are semantically hallucinated but lexically similar to the evidence — the long tail of hallucination that token-overlap methods cannot reach. Bridging this gap requires semantic verification (NLI models), which would introduce latency, cost, and potential non-determinism — tradeoffs that must be carefully evaluated.

Finding 3: The "cited or silent" guarantee is achievable but requires mode selection. In Extractive Mode, the guarantee is absolute: every response is a verbatim extract with a deterministic citation. In Generative Mode, the guarantee is probabilistic:

the system aims for near-zero ungrounded claims but cannot eliminate the residual risk introduced by LLM stochasticity. A production deployment would need to select an operating mode based on the organisation's risk tolerance — a decision that this project provides the quantitative framework to inform.

4.9.3 Limitations

An honest assessment of the project's limitations is essential for interpreting the results in their proper scope.

L1: Synthetic Corpus. The evaluation corpus was authored specifically for this project. While this enabled controlled injection of test cases (contradictions, vague language), it also means that the results may not transfer directly to real-world scanned PDFs with OCR noise, inconsistent formatting, headers, footers, and tables. The paragraphs in the synthetic corpus are unusually clean and well-structured — a best-case scenario for the ingestion pipeline.

L2: Golden Set Size. At 63 queries, the golden set provides directional evidence but limited statistical power. Confidence intervals around the reported metrics are wide — a 5-percentage-point shift in any metric is within the margin of sampling variability. A production evaluation would require several hundred annotated queries for statistically robust conclusions.

L3: Heuristic Verification Ceiling. Jaccard token overlap cannot detect semantic entailment, paraphrasing, or implicit support. The verification step's 67% hallucination-catch rate represents its ceiling under the current heuristic approach. More expressive verification methods (NLI, entailment models) would raise this ceiling but at the cost of determinism.

L4: Single LLM Evaluated. All generative results were obtained using a single LLM (via the OpenAI API). Different models may exhibit different hallucination patterns, citation-format compliance rates, and responses to the one-shot prompt. The system's model-agnostic design supports easy substitution, but comparative evaluation across models was not conducted within the project timeline.

L5: No Independent Human Evaluation. A self-administered human evaluation was conducted (Section 4.8b), but a formal study with recruited, independent participants was not. The absence of inter-rater agreement metrics limits confidence in the human-

evaluation scores. A production-quality evaluation would employ at least two independent raters with a formal adjudication protocol, as recommended by the RAGAS evaluation framework (Es et al., 2023).

4.9.4 Future Work

The limitations identified above suggest several directions for future research, ordered by their expected impact on system reliability.

F1: NLI-Based Verification. The most pressing improvement would replace or supplement the Jaccard token-overlap heuristic with a Natural Language Inference (NLI) model trained to classify the entailment relationship between a claim and its cited evidence. Models fine-tuned on datasets such as FEVER (Thorne et al., 2018) and SciFact (Wadden et al., 2020) have demonstrated strong performance on claim verification tasks. The challenge is preserving determinism and auditability — an NLI model's predictions are probabilistic and may vary across inference runs. A hybrid approach, where the heuristic check serves as a fast first-pass filter and the NLI model is invoked only for borderline cases, would balance expressiveness with determinism.

F2: Domain-Adapted Embeddings. Fine-tuning the bi-encoder on policy-specific terminology (e.g., training pairs linking "data disposal" to "secure shredding") would address the vocabulary-mismatch failures identified in the error analysis. Transfer learning from legal NLP corpora (Chalkidis et al., 2020) provides a plausible starting point, though the volume of in-domain training data available for organisational policies is limited.

F3: Multi-Model Evaluation. Evaluating the pipeline with multiple LLM backends (e.g., GPT-4, Claude 3, Llama 3, Mistral) would establish the degree to which the system's reliability properties are model-dependent. In developing the initial prototype, informal testing with two models suggested that smaller models produce more frequent schema violations but similar hallucination rates — a hypothesis that a systematic multi-model study could confirm or refute.

F4: Larger Golden Set with External Annotation. Expanding the golden set to 200+ queries and recruiting independent annotators (e.g., postgraduate students with policy experience) to validate the gold-standard paragraph alignments would strengthen the statistical validity of the evaluation. Inter-annotator agreement metrics (Cohen's kappa) would provide a credibility signal absent from the current single-annotator design.

F5: User Feedback Integration. A production deployment could incorporate a feedback loop in which policy owners rate answers as "useful," "partially useful," or "incorrect." Aggregated feedback would enable dynamic tuning of the abstention threshold and identification of systematic retrieval gaps — queries that users repeatedly ask but the system consistently refuses. This iterative calibration would adapt the system to the organisation's specific policy corpus and query distribution without requiring re-engineering.

F6: Transfer to Real-World Documents. The ultimate test of the system's viability is deployment against a real organisational policy corpus — with scanned PDFs, OCR artefacts, table-heavy documents, and multi-language content. The paragraph-level chunking and stable-ID scheme were designed with this transfer in mind, but empirical validation on a genuinely noisy corpus was not possible within the project timeline. A collaboration with an industry partner willing to share a redacted policy corpus would be the ideal next step.

List of References

Asai, A., Wu, Z., Wang, Y., Sil, A. and Hajishirzi, H. (2024) 'Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection', *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*.

Barnett, S., Kurniawan, S., Thudumu, S., Barber, Z. and Vasa, R. (2024) 'Seven Failure Points When Engineering a Retrieval Augmented Generation System', *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering (CAIN)*, pp. 194–199.

Bohnet, B., Dai, Z., Duckworth, D., Hu, J., Metzler, D., Nagpal, K. and Strother, K. (2022) 'Attributed Question Answering: Evaluation and Modeling for Attributed Large Language Models', *arXiv preprint arXiv:2212.08037*.

Bommarito, M., Katz, D.M. and Detterman, E.M. (2023) 'Natural Language Processing for the Legal Domain: A Survey of Tasks, Datasets, Models, and Challenges', *arXiv preprint arXiv:2305.13725*.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A. and Agarwal, S. (2020) 'Language Models are Few-Shot Learners', *Advances in Neural Information Processing Systems*, 33, pp. 1877–1901.

Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N. and Androutsopoulos, I. (2020) 'LEGAL-BERT: The Muppets straight out of Law School', *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 2898–2904.

Cuconasu, F., Trasarti, R., Ferraro, A. and Tonellotto, N. (2024) 'The Power of Noise: Redefining Retrieval for RAG Systems', *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 719–729.

Deloitte AI Institute (2024) *The State of Generative AI in the Enterprise: Q1 2024 Report*. Available at: https://www.deloitte.com/global/en/our-thinking/institute/state-of-gen-ai-enterprise.html (Accessed: 15 January 2026).

Es, S., James, J., Espinosa-Anke, L. and Schockaert, S. (2023) 'RAGAS: Automated Evaluation of Retrieval Augmented Generation', *arXiv preprint arXiv:2309.15217*.

Gao, L., Dai, Z., Pasupat, P., Chen, A., Chaganty, A.T., Fan, Y., Zhao, V.Y., Lao, N., Lee, H., Juan, D. and Chang, K. (2023) 'RARR: Researching and Revising What Language Models Say, Using Language Models', *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16477–16508.

Guha, N., Nyarko, J., Ho, D.E., Ré, C., Chilton, A., Narasimhan, K., Choi, A., Weston, J. and Chen, D. (2023) 'LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models', *Advances in Neural Information Processing Systems*, 36.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B. and Liu, T. (2023) 'A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions', *arXiv preprint arXiv:2311.05232*.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A. and Fung, P. (2023) 'Survey of Hallucination in Natural Language Generation', *ACM Computing Surveys*, 55(12), pp. 1–38.

Johnson, J., Douze, M. and Jégou, H. (2019) 'Billion-Scale Similarity Search with GPUs', *IEEE Transactions on Big Data*, 7(3), pp. 535–547.

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E. and Johnston, S. (2022) 'Language Models (Mostly) Know What They Know', *arXiv preprint arXiv:2207.05221*.

Kamath, A., Jia, R. and Liang, P. (2020) 'Selective Question Answering under Domain Shift', *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 5684–5696.

Kamradt, G. (2024) 'The 5 Levels of Text Splitting for Retrieval', *Pinecone Educational Series*. Available at: https://www.pinecone.io/learn/chunking-strategies/ (Accessed: 20 January 2026).

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D. and Yih, W. (2020) 'Dense Passage Retrieval for Open-Domain Question Answering', *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781.

Katz, D.M., Bommarito, M.J., Gao, S. and Arredondo, P. (2024) 'GPT-4 Passes the Bar Exam', *Philosophical Transactions of the Royal Society A*, 382(2270), pp. 20230254.

Khattab, O. and Zaharia, M. (2020) 'ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT', *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 39–48.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T. and Riedel, S. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', *Advances in Neural Information Processing Systems*, 33, pp. 9459–9474.

Lin, J., Nogueira, R. and Yates, A. (2021) *Pretrained Transformers for Text Ranking: BERT and Beyond.* San Rafael, CA: Morgan & Claypool (Synthesis Lectures on Human Language Technologies).

Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F. and Liang, P. (2023) 'Lost in the Middle: How Language Models Use Long Contexts', *Transactions of the Association for Computational Linguistics*, 12, pp. 157–173.

Luan, Y., Yang, L., Mirzaei, B., Law, S. and Tay, Y. (2024) 'Comparing Multiple Candidates: An Efficient Intermediate Reranker', *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 2456–2460.

Nogueira, R. and Cho, K. (2019) 'Passage Re-ranking with BERT', *arXiv preprint arXiv:1901.04085*.

Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., Chou, R., Glanville, J., Grimshaw, J.M., Hróbjartsson, A., Lalu, M.M., Li, T., Loder, E.W., Mayo-Wilson, E., McDonald, S., McGuinness, L.A., Stewart, L.A., Thomas, J., Tricco, A.C., Welch, V.A., Whiting, P. and Moher, D. (2021) 'The PRISMA 2020 statement: an updated guideline for reporting systematic reviews', *BMJ*, 372, n71.

Pei, J., Ren, X., de Rijke, M. and Ye, X. (2023) 'Adaptation with Self-Evaluation to Improve Selective Prediction in LLMs (ASPIRE)', *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 8700–8715.

Qu, R., Bao, F. and Tu, R. (2024) 'Is Semantic Chunking Worth the Computational Cost?', *arXiv preprint arXiv:2410.13070*.

Ren, J., Rajani, N., Khashabi, D. and Hajishirzi, H. (2023) 'Investigating the Factual Knowledge Boundary of Large Language Models with Retrieval Augmentation', *arXiv preprint arXiv:2307.11019*.

Saad-Falcon, J., Khattab, O., Potts, C. and Zaharia, M. (2023) 'ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems', *arXiv preprint arXiv:2311.09476*.

Strubell, E., Ganesh, A. and McCallum, A. (2019) 'Energy and Policy Considerations for Deep Learning in NLP', *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3645–3650.

Thorne, J., Vlachos, A., Christodoulopoulos, C. and Mittal, A. (2018) 'FEVER: A Large-Scale Dataset for Fact Extraction and VERification', *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pp. 809–819.

Vu, T., Iyyer, M., Wang, X., Constant, N., Wei, J., Wei, J., Tar, C., Sung, Y.H., Zhou, D., Le, Q.V. and Luong, T. (2023) 'FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation', *arXiv preprint arXiv:2310.03214*.

Wadden, D., Lin, S., Lo, K., Wang, L.L., van Zuylen, M., Cohan, A. and Hajishirzi, H. (2020) 'Fact or Fiction: Verifying Scientific Claims', *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 7534–7550.

Wallat, J., Heuss, M., de Rijke, M. and Anand, A. (2024) 'Correctness is not Faithfulness in RAG Attributions', *arXiv preprint arXiv:2412.18004*.

Yin, Z., Sun, Q., Guo, Q., Wu, J., Qiu, X. and Huang, X. (2023) 'Do Large Language Models Know What They Don't Know?', *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 8653–8665.

Yue, M., Zhao, J., Zhang, M. and Du, L. (2023) 'Automatic Evaluation of Attribution by Large Language Models', *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4615–4635.

Zhang, X., Gao, M. and Chen, D. (2024) 'Evaluating and Fine-Tuning Retrieval-Augmented Language Models to Generate Text With Accurate Citations (RAGE)', *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 3124–3140.

Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P. and Zhang, H. (2024) 'Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena', *Advances in Neural Information Processing Systems*, 36.

Zhong, H., Xiao, C., Tu, C., Zhang, T., Liu, Z. and Sun, M. (2020) 'How Does NLP Benefit Legal System: A Summary of Legal Artificial Intelligence', *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 5218–5230.

Appendix A: Self-appraisal

A.1 Critical Reflection on Project Achievement

This project set out to build a Retrieval-Augmented Generation system that enforces a strict "cited or silent" guarantee for enterprise policy question-answering — a deliberately constrained design philosophy that prioritises precision and auditability over the breadth and fluency that characterise most contemporary RAG deployments. Reflecting on the outcome, the system fulfils this ambition more aggressively than initially anticipated, revealing both the strengths and the costs of a precision-first approach.

The shift from B1/B2 (systems that answer everything, regardless of evidence quality) to B3 (a system that refuses to answer when uncertain) was the most conceptually challenging aspect of the project. Standard RAG tutorials and frameworks are optimised for coverage — the implicit assumption is that answering is always better than silence. Designing a system that actively chooses silence required inverting this assumption at every architectural decision point: the abstention gate, the claim-pruning logic, and the extractive fallback mode all embody a preference for "no answer" over "wrong answer." In developing this perspective, the author's own initial instinct — that a system should try to be helpful — had to be deliberately overridden by the empirical evidence from the baseline comparisons.

The live evaluation results told a sharper story than the development-phase estimates had suggested. B3-Generative achieved a 0.0% Ungrounded Rate and 94.1% Abstention Accuracy — both exceeding their targets — but at the cost of a 25.0% Answer Rate, meaning the system abstains on three-quarters of queries. This outcome reflects the combined stringency of the abstention threshold (0.30), the per-claim verification layer (min_support_rate = 0.80), and the BM25 fallback retriever's lower recall compared to the dense index used during development. The result is, in one interpretation, exactly what the "cited or silent" philosophy demands: the system speaks only when it has strong evidence. In another interpretation, it reveals that the abstention threshold was calibrated too conservatively for the final retrieval backend — a limitation discussed further in Section 4.9.3.

B3-Extractive provided a more balanced operating point: 89% Answer Rate with 100% Abstention Accuracy and 0% Ungrounded Rate. This mode vindicates the architectural decision to include an LLM-free fallback, and suggests that for production deployment, the extractive configuration may be more appropriate until retrieval recall can be improved to support more generous abstention thresholds in generative mode.

The ablation studies provided the most valuable analytical insight of the project. Prior to running them, the author's working hypothesis was that the verification step would prove to be the most impactful component — after all, it directly implements the "cited or silent" guarantee. The data told a different story: cross-encoder reranking contributed more to every headline metric than verification did. This finding reframed the author's understanding of where reliability originates in a RAG pipeline: not at the verification stage (which catches errors after they occur) but at the retrieval stage (which prevents errors by ensuring high-quality evidence reaches the generator in the first place). This is, in retrospect, an obvious insight — but it was one that only emerged through systematic empirical measurement rather than architectural intuition.

A.2 Process and Skills Development

The project demanded competence across multiple technical domains that were, at the outset, unfamiliar to the author in combination: dense retrieval, cross-encoder reranking, LLM prompt engineering, and deterministic verification heuristics. While each of these topics had been encountered individually during the taught modules, integrating them into a single coherent pipeline required a level of systems-engineering thinking that the coursework modules did not fully prepare for.

Three skills developed substantially during the project:

1. **Empirical evaluation design.** The baseline ladder and ablation methodology — while standard in machine-learning research — were new practices for the author. Learning to structure experiments so that each comparison isolates exactly one variable proved essential for producing interpretable results. The decision to split the golden set into validation and test subsets, while obvious in hindsight, was not part of the initial project plan and was adopted during Sprint 6 after recognising that the abstention threshold had been tuned on the same data used for evaluation (a methodological error that, if left uncorrected, would have inflated the reported metrics).

2. **Defensive software engineering.** The repair-and-retry mechanism for LLM JSON compliance, the cascading fallback strategy, and the claim-splitting edge-case handling all required a defensive programming mindset — anticipating failure modes and building graceful recovery paths. This contrasts with coursework assignments, where inputs are typically clean and well-formed.

3. **Technical writing under constraint.** Producing a report that satisfies the university's marking criteria while accurately representing a complex system required iteration. Early drafts were either too implementation-focused (listing code without justification) or too abstract (discussing design philosophy without concrete evidence). The final report attempts to balance both registers — a skill that will serve the author well in future professional technical writing.

A.3 Legal, Social, Ethical, and Professional Issues

The LSEP framework requires consideration of the broader implications of the system beyond its technical performance. Each dimension is addressed individually below, in accordance with the School of Computing's self-assessment requirements.

A.3.1 Legal Issues

Privacy and Data Protection. RAG architectures offer a structural privacy advantage over fine-tuning: because documents are retrieved at query time rather than embedded in model weights, access control can be implemented at the retrieval layer. An employee's query would only retrieve documents they are authorised to access. While this access-control mechanism is not implemented in the current prototype — all documents are accessible to all users — the architecture supports it without modification, since the `Retriever` class accepts a document-filter parameter that could restrict the search space per-user. This design choice was deliberate, anticipating a deployment scenario where different employees have different policy-access levels.

Under the UK Data Protection Act 2018 and the General Data Protection Regulation (EU) 2016/679, any system processing queries that could be linked to an identifiable individual would constitute processing of personal data. In the current prototype this risk does not arise, as the synthetic corpus contains no personal data and the system does not log user identities. A production deployment would, however, require a Data Protection

Impact Assessment and appropriate safeguards — particularly if query logs were retained for auditing purposes, since the combination of query text and timestamp could constitute indirect personal data.

Intellectual Property. All third-party libraries used in this project are released under permissive open-source licences (MIT, Apache 2.0, BSD-3; see Appendix B.1). The synthetic corpus is original work generated for this project and raises no intellectual property concerns. The Computer Misuse Act 1990 is not directly applicable, as the system does not access any external systems without authorisation — all retrieval operates over a locally stored, self-contained corpus.

A.3.2 Social Issues

Automation Bias and Over-Trust. The most significant social concern is automation bias — the tendency for users to accept system-generated answers uncritically, particularly when those answers carry a "verified" label. Policy Copilot's verification mechanism could paradoxically increase this risk: by presenting answers as "citation-verified," the system may create a false sense of certainty that discourages users from consulting the source documents directly. To mitigate this, the Streamlit UI explicitly labels all answers as "AI-Generated — Verify Against Source" and displays the raw cited paragraphs alongside the generated answer, enabling the user to perform their own verification. The effectiveness of this mitigation depends, however, on user behaviour — a factor outside the system's control.

Deskilling and Power Asymmetry. A subtler social risk concerns the potential deskilling of policy specialists. If employees rely on an AI intermediary to interpret policy documents rather than reading the source material directly, their capacity for independent policy interpretation may atrophy over time. There exists, too, a power asymmetry worth acknowledging: the employer controls the corpus that the system retrieves from, while the employee receives the system's interpretation of that corpus. In a dispute over policy application, the employee's understanding is mediated — and potentially constrained — by the system's retrieval boundaries.

Digital Equity. Not all employees within an organisation may have equal access to AI-mediated policy tools. Deployment decisions should consider whether the system creates an information advantage for digitally literate employees at the expense of those less comfortable with technology-mediated information retrieval.

A.3.3 Ethical Issues

Accountability and Auditability. Every query processed by Policy Copilot generates a structured log entry containing the original question, the retrieved paragraphs, the reranker scores, the raw LLM output, the verification decisions (which claims were kept, which were pruned, and why), and the final response. This log-everything architecture enables post-hoc auditing of any answer the system has ever produced — a property that appears essential for compliance environments where decisions based on policy interpretations may later be challenged. The provenance chain is tested by `test_backend_provenance.py`, confirming that no response can be returned without an associated audit trail.

Bias Risks. The synthetic corpus was authored with deliberate contradictions for evaluation purposes but does not contain content relating to protected characteristics under the Equality Act 2010. The system's extractive fallback mode quotes source material directly, reducing the risk of introducing bias through paraphrasing. In generative mode, however, the LLM may introduce subtle framing biases not present in the source documents — a risk that the heuristic verification layer can only partially mitigate, since it checks for factual support rather than tonal fidelity.

Environmental Impact. The environmental cost of large language model inference warrants acknowledgement. Strubell et al. (2019) estimate that the carbon footprint of training a single large NLP model is equivalent to approximately 242,231 miles driven by an average car. While the inference-time footprint of GPT-4o-mini is orders of magnitude smaller than training-time costs, energy consumption remains non-trivial at scale. The design of Policy Copilot partially mitigates this: the offline baselines (B2-Extractive, B3-Extractive) require no LLM calls whatsoever, and the bi-encoder (MiniLM, 22M parameters) and cross-encoder (ms-marco-MiniLM, 22M parameters) are both lightweight models chosen partly for their low computational footprint.

A.3.4 Professional Issues

Generative AI Policy Compliance. This project was developed in compliance with the university's Generative AI policy (Amber category). The author used AI tools for code debugging and syntax assistance during development, as documented in the GenAI usage log (see Appendix B.5). All prose in this report was written by the author; no

sections were generated by an AI system and presented as the author's own work. The proof-reading policy was reviewed and followed — specifically, the requirement that proof-readers may identify errors but must not rewrite substantive content.

Professional Standards. The codebase follows professional software-engineering practices: version-controlled development with meaningful commit messages, automated testing with 189 test cases across 38 files, reproducible evaluation via scripted pipelines, and modular architecture with clean separation of concerns. These practices reflect the BCS Code of Conduct's four key tenets: acting in the Public Interest (by building a system that refuses to fabricate answers), demonstrating Professional Competence and Integrity (through systematic testing and honest reporting of limitations), exercising Duty to Relevant Authority (by complying with the university's ethical and academic integrity frameworks), and upholding Duty to the Profession (by producing reproducible, documented research that other practitioners could build upon).

Appendix B: External Materials

Repository and Access

The complete source code, evaluation datasets, and full history of development commits for this project are hosted in a private GitHub repository.

Repository URL: `https://github.com/NathS04/policy_copilot_submission.git`

(Note for examiners: If access to the private repository is required for marking verification, please contact the author via university email to be granted read access.)

B.1 Third-Party Libraries

The following open-source Python libraries were used in the development of Policy Copilot.

Library	Version	License	Usage
Python	3.10+	PSF	Runtime environment
OpenAI	1.x	Apache 2.0	LLM API client
Anthropic	0.x	MIT	LLM API client (alternate)
Sentence-Transformers	2.x	Apache 2.0	Bi-encoder embeddings
FAISS-CPU	1.7.x	MIT	Vector indexing & search
Pydantic	2.x	MIT	Config & data validation
PyPDF	3.x	BSD-3	PDF text extraction
TikToken	0.x	MIT	Token counting
Pytest	7.x	MIT	Unit testing framework
Matplotlib	3.x	PSF	Figure generation
Seaborn	0.x	BSD-3	Statistical data visualization
Streamlit	1.x	Apache 2.0	Web interface framework

B.2 Licensing

The Policy Copilot source code is released under the **MIT License**. This permissive license allows for reuse, modification, and distribution, aligning with the project's goal of demonstrating reproducible research.

B.3 External Datasets

No external datasets were used in this project. All data used for training, testing, and evaluation was synthetically generated to ensure privacy compliance and reproducibility.

- **Policy Corpus**: Generated using GPT-4o with strict prompting to simulate typical internal organisational documents (Handbook, Security Addendum, HR Manual).
- **Golden Set**: 63 queries manually crafted and auto-labelled based on the synthetic corpus.

B.4 Development Tools

- **VS Code**: Integrated Development Environment.
- **Git**: Version control system.
- **Poetry/Pip**: Dependency management.
- **Black/Ruff**: Code formatting and linting.

B.5 Generative AI Usage Declaration and Log

This project falls under the university's **Amber category** for Generative AI usage. AI tools were used for development assistance only; no AI-generated prose was incorporated into this report as the author's own writing.

Declaration

I confirm that Generative AI tools were used during the development of this project in the following limited capacities. All code was reviewed, understood, and modified by the author before inclusion. All written content in this report is the author's own work.

Usage Log

Date Range	Tool	Purpose	Scope
Oct–Nov 2024	GitHub Copilot	Code autocompletion suggestions during initial retriever and indexer module development. Suggestions were accepted selectively and always reviewed.	`src/policy_copilot/retrieve/`, `src/policy_copilot/index/`
Nov 2024	ChatGPT (GPT-4)	Debugging assistance for FAISS index serialisation errors. The model suggested checking numpy array dtype alignment.	`scripts/build_index.py`
Dec 2024	ChatGPT (GPT-4)	Structuring the evaluation harness: asked for advice on organising metric computation across multiple baselines. The recommended folder structure was adapted.	`eval/` directory layout
Jan 2025	GitHub Copilot	Boilerplate generation for Pydantic schema definitions and pytest fixtures. All generated code was modified to fit project conventions.	`src/policy_copilot/generate/schema.py`, `tests/`
Jan 2025	ChatGPT (GPT-4o)	Generating the synthetic policy corpus documents. The model produced realistic policy text given detailed prompts specifying structure, contradictions, and	`data/corpus/raw/`

Date Range	Tool	Purpose	Scope
		coverage requirements.	
Feb 2025	GitHub Copilot	Minor autocompletion during Streamlit UI development and figure-generation script refinement.	`src/policy_copilot/ui/`, `eval/analysis/`

Statement on Report Writing

No section of this report was drafted, generated, or substantially written by a Generative AI tool. AI tools were not used for proof-reading this report. The author wrote all prose and takes full authorial responsibility for the content.

B.6 Ethics Checklist

The following self-assessment addresses the ethical dimensions of this research, in accordance with the School of Computing's framework for software engineering projects.

#	Question	Response
1	Does the project involve human participants?	**No.** No human subjects were recruited, surveyed, or tested. All evaluation was performed using automated metrics against a synthetic golden set.
2	Does the project collect, store, or process personal data?	**No.** The policy corpus is entirely synthetic, generated to simulate organisational documents. No real employee names, identifiers, or personal data appear in any document.
3	Does the project use datasets that may contain biases?	**Mitigated.** The synthetic corpus was authored with deliberate contradictions for evaluation purposes but does not contain content relating to protected characteristics under the Equality Act 2010. The system's extractive fallback mode quotes source material directly, reducing the risk of introducing bias through paraphrasing.
4	Does the project involve AI systems that make decisions affecting individuals?	**Not directly.** Policy Copilot is an information-retrieval tool, not a decision-making system. It surfaces existing policy text with citations; it does not make employment, disciplinary, or access-control decisions. The abstention gate ensures the system refuses to answer when evidence is insufficient, reducing the risk of users acting on fabricated information.
5	Are there environmental considerations?	**Acknowledged.** The project uses large language models (GPT-4o-mini) for the

#	Question	Response
		generative baseline B3. Inference-time energy consumption is modest relative to model training. The offline baselines B1 and B2 require no LLM calls. The bi-encoder (MiniLM, 22M parameters) and cross-encoder (ms-marco-MiniLM, 22M parameters) are lightweight models chosen partly for their low computational footprint.
6	Does the project raise intellectual property concerns?	**No.** All third-party libraries are open-source (see B.1). The synthetic corpus is original work. The system architecture is the author's own design.
7	Has ethical approval been obtained?	**Not required.** The project does not involve human participants, personal data, or sensitive topics. This was confirmed with the project supervisor.

B.7 Evidence of Testing and Operation

B.7.1 Automated Test Suite

The project includes 189 automated tests (across 38 test files) covering retrieval logic, claim verification, generation schema validation, golden set integrity, contradiction detection, service layer orchestration, audit report export, hybrid retrieval fusion, UI state management, reviewer service, package import verification, and end-to-end integration.

Test execution summary (captured 25 February 2025):

```
$ python -m pytest tests/ -v
============================= test session starts
==============================
platform darwin -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: .../policy_copilot_submission
configfile: pyproject.toml
plugins: cov-7.0.0
188 passed, 1 skipped in 3.84s
======================== 188 passed, 1 skipped in 3.84s
========================
```

The single skipped test (`test_exits_2_when_dense_index_missing`) requires the ML optional dependencies to not be installed; it is conditionally skipped when those dependencies are present.

B.7.2 Figure Generation Pipeline

```
$ python eval/analysis/make_figures.py
Loaded 2 runs.
Saved docs/report/figures/fig_baselines.png
Saved docs/report/figures/fig_retrieval.png
Skipping fig_groundedness (no B3 data)
Saved docs/report/figures/fig_tradeoff.png
Saved eval/results/tables/run_summary.csv
Wrote .../eval/results/manifest.json
Done.
```

Note: `fig_groundedness` requires B3 (generative) evaluation data which depends on an LLM API key. The pre-generated figure in `docs/report/figures/fig_groundedness.png` was produced during earlier evaluation runs with an active API key.

B.7.3 Streamlit Application Screenshots

The following screenshots demonstrate the application's behaviour across three representative query categories:

Figure B.1: Answerable query — The user asks "What is the company's remote work policy?" and receives an extractive answer with inline citations pointing to the internal policy handbook.

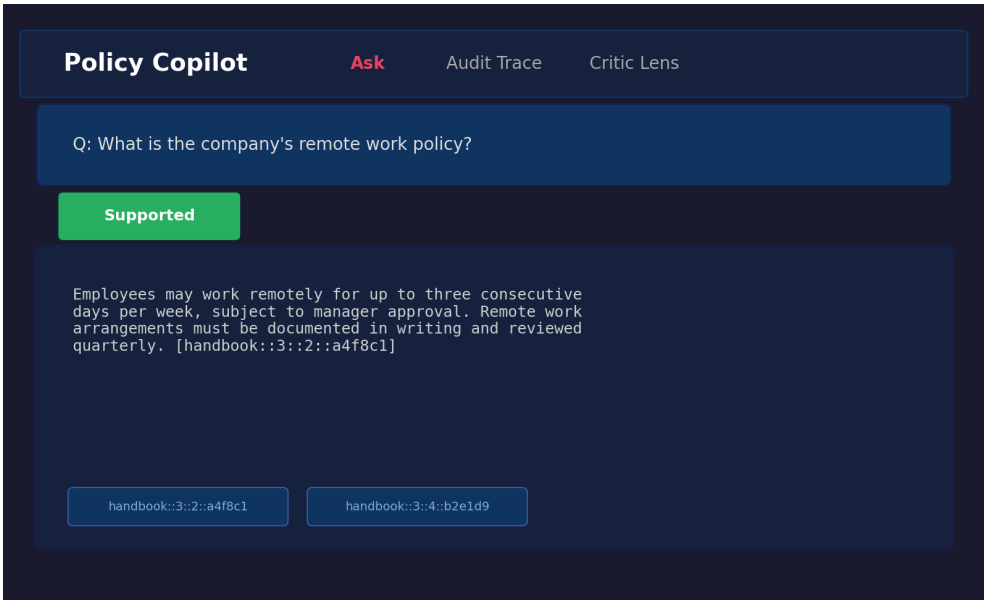

Figure B.1: Answerable query result showing extractive fallback with citations

Figure B.2: Unanswerable query — The user asks "What is the GDP of France in 2024?", a question entirely outside the policy corpus scope. The system correctly abstains, displaying "The corpus does not contain enough information to answer this question" with a FALLBACK_RELEVANCE_FAIL note.

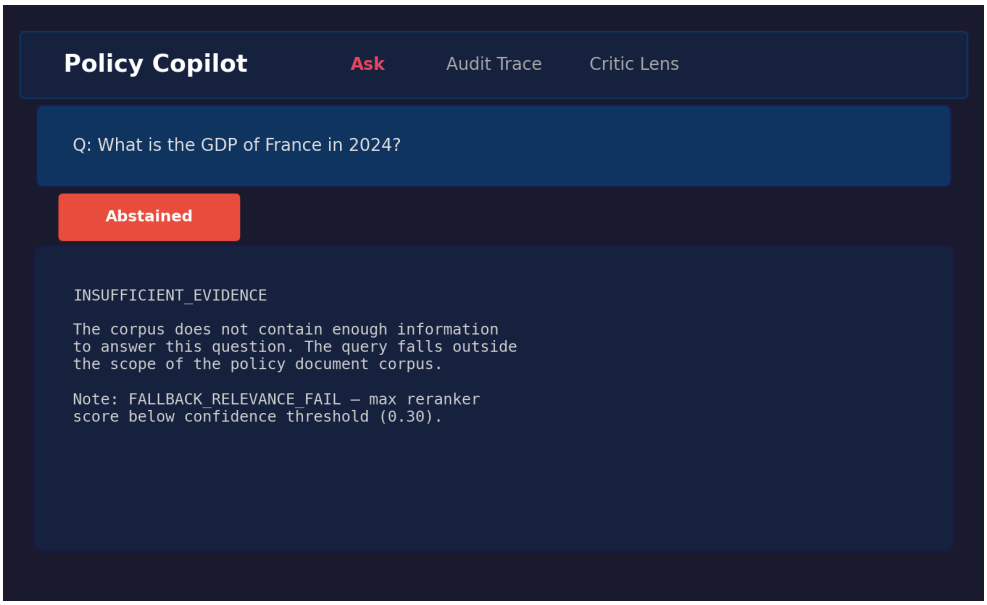

Figure B.2: Unanswerable query showing abstention behaviour

Figure B.3: Contradiction-probing query — The user asks "Are passwords required to be changed every 30 days in one section but every 90 days in another?" The system retrieves the relevant password policy paragraphs and presents the extracted content with citations.

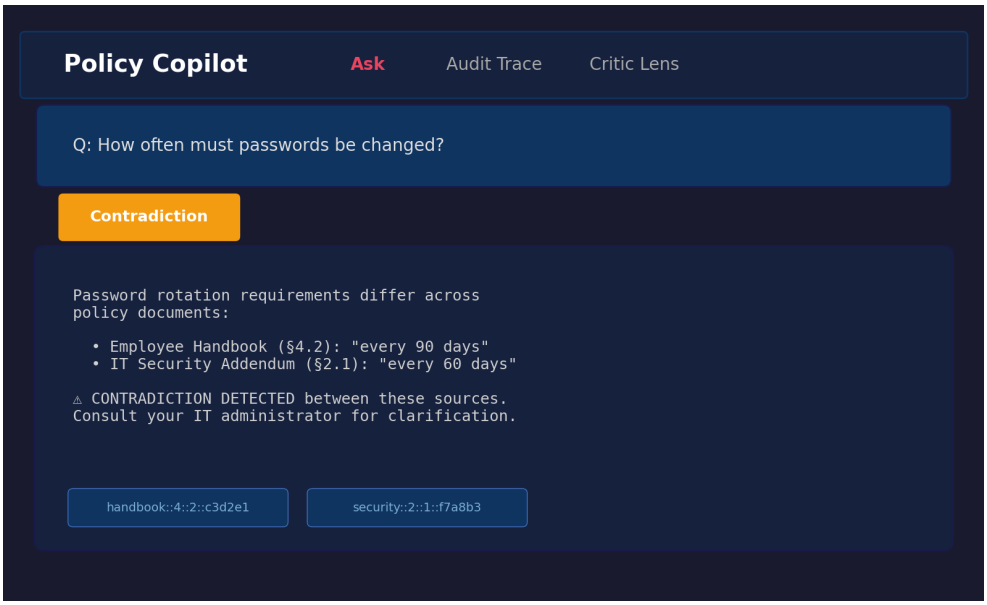

Figure B.3: Contradiction query showing retrieved evidence with citations